Kunpeng HPCKit 26.1.RC1

迁移指南

文档版本	01
发布日期	2026-06-30



华为技术有限公司

版权所有 © 华为技术有限公司 2026。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明


和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

安全声明

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
https://www.huawei.com/cn/psirt/vul-response-process
如企业客户须获取漏洞信息，请参见如下网址：
https://securitybulletin.huawei.com/enterprise/cn/security-advisory

 版权所有 © 华为技术有限公司

目 录

1 概述.....1
2 Intel MKL 迁移到 KML.....3
2.1 如何识别使用了 Intel MKL.....3
2.2 向量数学库（VML）迁移.....3
2.3 基础线性代数库（BLAS）迁移.....5
2.4 稀疏基础线性代数库（Sparse BLAS）迁移.....8
2.5 快速傅里叶变换库（FFT）迁移.....11
2.6 线性代数运算库（LAPACK）迁移.....18
2.7 迭代求解器（ITERATIVE SOLVER）迁移.....24
2.8 直接求解器（DIRECT SOLVER）迁移.....26
2.9 线性代数运算库（SCALAPACK）迁移.....32
2.10 短向量数学库（SVML）迁移.....33
3 其他常用数学库迁移到 KML.....35
3.1 LIBM、MATHLIB、LIBIMF 迁移到 KML_MATH.....35
3.2 OpenBLAS 迁移到 KML_BLAS.....37
3.3 FFTW 迁移到 KML_FFT.....37
3.4 Netlib LAPACK 迁移到 KML_LAPACK.....38
3.5 PETSc 迁移到 KML_SOLVER.....46
3.5.1 单机版.....46
3.5.2 集群版.....48
3.6 HYPRE 迁移到 KML_SOLVER.....51
3.6.1 集群版.....51
3.7 MUMPS 迁移到 KML_SOLVER.....52
3.7.1 集群版.....52
3.8 Netlib SCALAPACK 迁移到 KML_SCALAPACK.....55
3.9 Eigen 库使用 KML.....56
3.10 Numpy 使用 KML.....60
4 修订记录.....62

 版权所有 © 华为技术有限公司

1 概述

本文指导用户如何将Intel MKL或一些常用的数学库迁移至鲲鹏数学库以发挥鲲鹏平台更优性能。

HPCKit组件中，仅数学库涉及迁移。

广义上的数学库是指提供高性能标量、矢量计算的数学函数接口库，常见的数学库有英特尔数学内核库，以下简称Intel MKL（Intel Math Kernel Library）及一些开源数学库。鲲鹏数学库，以下简称KML（Kunpeng Math Library）提供了基于鲲鹏平台优化的高性能数学函数。当用户将使用数学库函数的应用程序迁移到鲲鹏计算平台时，由于不同计算平台或不同数学库的函数定义差别，需要对程序重新适配开发。针对该问题，HPCKit提供了HPCKit迁移指南。

本指南包含将Intel MKL中的一些数学库迁移至KML的步骤（请参见**2.1 如何识别使用了Intel MKL**），包括MKL的VML至KML_VML，MKL的BLAS至KML_BLAS等，同时也包含一些常用数学库迁移至KML的步骤。通过迁移，用户可以在鲲鹏计算平台获得比一般开源数学库更优的计算性能。

迁移前后的数学库映射关系如**表1-1**所示。

表 1-1 迁移前后的数学库映射关系

被迁移数学库		**KML**	
名称	头文件	名称	头文件
icc-imf	math.h或mathimf.h	KML_MATH	km.h
MKL-VML	mkl.h或mkl_vml.h	KML_VML	kvml.h
MKL-BLAS	mkl.h或mkl_cblas.h	KML_BLAS	kblas.h
MKL-SPBLAS	mkl.h或mkl_spblas.h	KML_SPBLAS	kspblas.h
MKL-FFT	mkl.h或mkl_dfti.h	KML_FFT	kfft.h
MKL-LAPACK	mkl.h或mkl_lapack.h	KML_LAPACK	klapack.h，kservice.h
MKL-SOLVER	mkl.h或mkl_rci.h	KML_SOLVER（迭代法）	kml_iss.h

版权所有 © 华为技术有限公司

被迁移数学库		KML	
MKL-PARDISO或PARDISO-cluster	mkl_pardiso.h或mkl_cluster_sparse_solver.h	KML_SOLVER（直接法）	kml_scadss.h
PETSc	petscksp.h	KML_SOLVER（迭代法）	kml_iss.h/kml_scaiss.h
MUMPs	dmumps_c.h，smumps_c.h，cmumps_c.h或zmumps_c.h	KML_SOLVER（直接法）	kml_scadss.h
OpenBLAS	cblas.h	KML_BLAS	kblas.h
FFTW	fftw3.h	KML_FFT	kfft.h
MKL-SCALAPACK	mkl.h或mkl_scalapack.h	KML_SCALAPACK	kscalapack.h，klapack.h，kservice.h

版权所有 © 华为技术有限公司

2 Intel MKL 迁移到 KML

2.1 如何识别使用了 Intel MKL

将Intel MKL迁移至KML之前需要识别是否使用到Intel MKL，本文提供头文件、函数名和库文件3种识别方法。

Intel MKL为集成的数学库，其中包括了VML、BLAS、SPBLAS、FFT（DFT）、LAPACK等数学库的函数接口，所以在迁移过程中，需要识别具体使用到的库。例如，若使用了MKL中多个不同数学库的接口，则在迁移过程中需要包含所有用到的数学库的KML对应头文件。因为涉及到x86架构迁移到ARM架构，本指南只包含特定数学库的迁移方法，其余请参见**鲲鹏服务器代码移植参考手册**。

- 头文件识别

  查看源码中头文件是否包含mkl.h或其他带有MKL前缀的头文件。

- 函数名识别

  a. 通过perf命令抓取热点函数信息，寻找其中MKL的接口部分。

  b. 全局搜索源代码，根据MKL的函数名定位调用部分。

- 库文件识别

  a. 查看系统目录/opt/intel下是否有mkl文件夹。

  b. 查看代码编译文件Makefile，查看链接选项是否包括mkl的动态库（-lmkl_xxx）。

  说明

  实际场景中可以灵活组合以上方法进行使用。

2.2 向量数学库（VML）迁移

可替换性

KML_VML与MKL的VML所有接口入参完全一致，两者之间仅函数名存在差异，具体差异如**表2-1**所示。

 版权所有 © 华为技术有限公司

表 2-1 VML 的替换映射关系

MKL接口	KML对应接口	数据类型	描述
v?Add	v?add	s，d	加法函数。
v?Sub	v?sub	s，d	减法函数。
v?Sqr	v?sqr	s，d	平方函数。
v?Mul	v?mul	s，d	乘法函数。
v?Div	v?div	s，d	除法函数。
v?Sqrt	v?sqrt	s，d	开方函数。
v?Pow	v?pow	s，d	幂函数。
v?Exp	v?exp	s，d	以自然对数为底的指数函数。
v?Ln	v?ln	s，d	以自然对数为底的对数函数。
v?Log10	v?log10	s，d	以10为底的对数函数。
v?Cos	v?cos	s，d	余弦函数。
v?Sin	v?sin	s，d	正弦函数。
v?Tan	v?tan	s，d	正切函数。
v?Atan	v?atan	s，d	反正切函数。
v?Atan2	v?atan2	s，d	两个变量四个象限下的反正切函数。
v?Sincos	v?sincos	s，d	正弦函数与余弦函数。
v?Sinh	v?sinh	s，d	双曲正弦函数。
v?Cosh	v?cosh	s，d	双曲余弦函数。
v?Tanh	v?tanh	s，d	双曲正切函数。
v?Asinh	v?asinh	s，d	双曲反正弦函数。
v?Acosh	v?acosh	s，d	双曲反余弦函数。
v?Atanh	v?atanh	s，d	双曲反正切函数。

C 语言迁移步骤

针对KML_VML提供的接口，编译时直接替换库，上层软件调用的函数名进行相应修改适配，即可正常调用。

 版权所有 © 华为技术有限公司

Fortran 接口迁移步骤

Fortran接口迁移与**C语言迁移步骤**一致。

2.3 基础线性代数库（BLAS）迁移

可替换性

KML_BLAS对外提供的接口，与MKL-BLAS的接口参数和函数名完全一致，可以直接替换库使用。

如**表2-2**、**表2-3**、**表2-4**和**表2-5**所示为可替换的接口与MKL接口的映射关系。

表 2-2 Level1 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
cblas_?asum	cblas_?asum	s、d、c、z	向量元素绝对值求和。
cblas_?axpy	cblas_?axpy	s、d、c、z	向量缩放与加和。
cblas_?copy	cblas_?copy	s、d、c、z	向量拷贝。
cblas_?dot	cblas_?dot	s、d	向量点积。
cblas_?sdot	cblas_?sdot	d、sd	扩展精度点积。
cblas_?dotc_sub	cblas_?dotc_sub	c、z	共轭向量与另一向量的点积。
cblas_?dotu_sub	cblas_?dotu_sub	c、z	复数向量点积。
cblas_?nrm2	cblas_?nrm2	s、d、sc、dz	计算欧几里得范数。
cblas_?rot	cblas_?rot	s、d、cs、zd	基于平面上点旋转。
cblas_?rotg	cblas_?rotg	s、d	构造旋转平面。
cblas_?rotm	cblas_?rotm	s、d	修改旋转平面。
cblas_?rotmg	cblas_?rotmg	s、d	构造修改的转换平面。
cblas_?scal	cblas_?scal	s、d、c、z、cs、zd	向量缩放。
cblas_?swap	cblas_?swap	s、d、c、z	两个向量元素交换。
cblas_i?amin	cblas_i?amin	s、d、c、z	向量中最小绝对值的索引。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
cblas_i?amax	cblas_i?amax	s、d、c、z	向量中最大绝对值的索引。

表 2-3 Level2 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
cblas_?gbmv	cblas_?gbmv	s、d、c、z	向量与带状矩阵乘积。
cblas_?gemv	cblas_?gemv	s、d、c、z	向量与矩阵乘积。
cblas_?ger	cblas_?ger	s、d	一般矩阵秩1更新。
cblas_?gerc	cblas_?gerc	c、z	一般复数矩阵秩1更新。
cblas_?geru	cblas_?geru	c、z	一般共轭矩阵秩1更新。
cblas_?hbmv	cblas_?hbmv	c、z	向量与埃尔米特带状矩阵乘积。
cblas_?hemv	cblas_?hemv	c、z	向量与埃尔米特矩阵乘积。
cblas_?her	cblas_?her	c、z	复数埃尔米特矩阵秩1更新。
cblas_?her2	cblas_?her2	c、z	复数埃尔米特矩阵秩2更新。
cblas_?hpmv	cblas_?hpmv	c、z	向量与三角展开的埃尔米特矩阵乘积。
cblas_?hpr	cblas_?hpr	c、z	三角展开的埃尔米特矩阵秩1更新。
cblas_?hpr2	cblas_?hpr2	c、z	三角展开的埃尔米特矩阵秩2更新。
cblas_?sbmv	cblas_?sbmv	s、d	向量与对称带状矩阵乘积。
cblas_?spmv	cblas_?spmv	s、d	向量与压缩的对称矩阵乘积。
cblas_?spr	cblas_?spr	s、d	对称展开矩阵秩1更新。
cblas_?spr2	cblas_?spr2	s、d	对称展开矩阵秩2更新。
cblas_?symv	cblas_?symv	s、d	向量与对称矩阵乘积。
cblas_?syr	cblas_?syr	s、d	对称矩阵秩1更新。
cblas_?syr2	cblas_?syr2	s、d	对称矩阵秩2更新。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
cblas_?tbmv	cblas_?tbmv	s、d、c、z	向量与三角带状矩阵乘积。
cblas_?tbsv	cblas_?tbsv	s、d、c、z	线性方程组求解，稀疏矩阵为三角带状矩阵。
cblas_?tpmv	cblas_?tpmv	s、d、c、z	向量与压缩的三角矩阵乘积。
cblas_?tpsv	cblas_?tpsv	s、d、c、z	线性方程组求解，稀疏矩阵是压缩的三角矩阵。
cblas_?trmv	cblas_?trmv	s、d、c、z	向量与三角矩阵乘积。
cblas_?trsv	cblas_?trsv	s、d、c、z	线性方程组求解，系数矩阵是三角矩阵。

表 2-4 Level3 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
cblas_?gemm	cblas_?gemm	s、d、c、z	一般矩阵乘矩阵。
cblas_?hemm	cblas_?hemm	c、z	复数埃尔米特矩阵乘矩阵。
cblas_?her2k	cblas_?her2k	c、z	复数矩阵乘矩阵秩2更新，输出矩阵C为复数埃尔米特矩阵。
cblas_?herk	cblas_?herk	c、z	复数矩阵乘矩阵秩k更新，输出矩阵C为复数埃尔米特矩阵。
cblas_?symm	cblas_?symm	s、d、c、z	实数对称矩阵乘矩阵。
cblas_?syrk	cblas_?syrk	s、d	矩阵乘矩阵秩k更新，输出矩阵C为对称矩阵。
cblas_?syr2k	cblas_?syr2k	s、d	矩阵乘矩阵秩2更新，输出矩阵C为对称矩阵。
cblas_?trmm	cblas_?trmm	s、d、c、z	矩阵乘矩阵，其中一个矩阵为三角矩阵。
cblas_?trsm	cblas_?trsm	s、d、c、z	三角矩阵方程求解。

版权所有 © 华为技术有限公司

表 2-5 Extension 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
cblas_?axpby	cblas_?axpby	s、d、c、z	向量分别缩放与加和。
cblas_?gemm3m	cblas_?gemm3m	c、z	一般复数矩阵乘矩阵。

C 语言迁移步骤

针对KML_BLAS提供的接口，编译时直接将Intel MKL的BLAS的链接选项替换为KML_BLAS的链接选项，不需要做其他修改。

Fortran 接口迁移步骤

Fortran接口迁移与**C语言迁移步骤**一致。

涉及C/C++和Fortran语言混合编程时，为了保证链接成功，需要在C代码中对在混合编程中进行交换信息的全局变量和函数名后缀“_”或者使用编译选项-fno-underscoring，否则会出现链接报错。

2.4 稀疏基础线性代数库（Sparse BLAS）迁移

可替换性

KML_SPBLAS对外提供的接口，与MKL-SPBLAS的接口参数和函数名不一致，需要修改部分源码。

如**表2-6**、**表2-7**所示为可替换的接口与MKL接口的映射关系。

表 2-6 Level1 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
cblas_?axpyi	kml_sparse_?axpyi	s，d，c，z	稀疏向量分别缩放与加和
cblas_?doti	kml_sparse_?doti	s，d	稀疏向量点积。
cblas_?dotci	kml_sparse_?dotci_sub	c，z	共轭稀疏向量与另一稀疏向量的点积。
cblas_?dotui	kml_sparse_?dotui_sub	c，z	复数稀疏向量点积。
cblas_?gthr	kml_sparse_?gthr	s，d，c，z	将full-storage格式的稀疏向量中指定的元素加载到compressed格式的向量中

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
cblas_?gthrz	kml_sparse_?gthrz	s，d，c，z	将full-storage格式的稀疏向量y中指定的元素加载到compressed格式的向量中。
cblas_?roti	kml_sparse_?roti	s，d	基于平面上点旋转。
cblas_?sctr	kml_sparse_?sctr	s，d，c，z	将compressed格式的向量写入full-storage格式的稀疏向量

表 2-7 Level2、3 接口的替换映射关系

MKL接口	KML对应接口	数据类型	描述
mkl_?csrgemv	kml_sparse_?csrgemv	s，d，c，z	矩阵与向量计算，稀疏矩阵采用CSR格式存储。
mkl_?csrsymv	kml_sparse_?csrsymv	s，d，c，z	矩阵与向量计算，稀疏矩阵采用CSR格式存储。
mkl_cspblas_?csrgemv	kml_csparse_?csrgemv	s，d，c，z	矩阵与向量计算，稀疏矩阵采用CSR格式存储。
mkl_cspblas_?csrsymv	kml_csparse_?csrsymv	s，d，c，z	矩阵与向量计算，稀疏矩阵采用CSR格式存储。
mkl_?csrmv	kml_sparse_?csrmv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式的稀疏矩阵。
mkl_?csrsv	kml_sparse_?csrsv	s，d，c，z	求解三角矩阵方程组计算，稀疏矩阵采用CSR格式存储。
mkl_?csrmm	kml_sparse_?csrmm	s，d，c，z	矩阵与矩阵计算，其中一个稀疏矩阵采用CSR格式存储。
mkl_?csradd	kml_sparse_?csradd	s，d，c，z	计算2个CSR格式的稀疏矩阵之和。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
mkl_?csrmultcsr	kml_sparse_?csrmultcsr	s，d，c，z	矩阵与矩阵计算，其中3个稀疏矩阵采用3-array CSR格式存储。
mkl_?csrmultd	kml_sparse_?csrmultd	s，d，c，z	矩阵与矩阵计算，其中2个稀疏矩阵采用CSR格式存储。

C 语言迁移步骤

需要迁移的部分主要有三大类：

- 函数名
  可替换的接口与MKL接口的映射关系请参见**可替换性**。
- 函数入参
  部分接口的参数再MKL中使用指针进行传递，KML接口直接传入数值，具体请参见**《 Kunpeng HPCKit 26.1.RC1 开发指南 》**中“鲲鹏数学库 开发指南”章节。
- 声明变量
  MKL部分变量带有MKL前缀，需要改为KML。

例：csrmv接口

迁移前：

```
#include "mkl_spblas.h"

char transa = 'N';
MKL_INT m = 4;
MKL_INT k = 4;
float alpha = 0.5;
float beta = 1.2;
char *matdescra = "G00F"; // 一般矩阵，基 1 索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
MKL_INT indx[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
MKL_INT pntrb[4] = {1, 4, 6, 9};
MKL_INT pntre[4] = {4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
mkl_scsrmv(&transa, &m, &k, &alpha, matdescra, val, indx, pntrb, pntre, x, &beta, y);
return 0;
/*
*  Output Y:
*
*     12.80  -14.80  26.50   8.10
*
* */
```

迁移后：

```
#include "kspblas.h"

kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
KML_INT k = 4;
float alpha = 0.5;
```


版权所有 © 华为技术有限公司

```
float beta = 1.2;
char *matdescra = "G00F"; // 一般矩阵，基 1 索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT indx[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
KML_INT pntrb[4] = {1, 4, 6, 9};
KML_INT pntre[4] = {4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_scsrmv(opt, m, k, alpha, matdescra, val, indx, pntrb, pntre, x, beta,
y);
return status;
/*
*  Output Y:
*
*     12.80  -14.80  26.50   8.10
*
* */
```

Fortran 接口迁移步骤

Fortran接口迁移与**C语言迁移步骤**一致。

2.5 快速傅里叶变换库（FFT）迁移

可替换性

由于KML_FFT与MKL接口并不一致，需要根据代码逻辑替换代码，具体使用方法请参见《**Kunpeng HPCKit 26.1.RC1 开发指南**》中“鲲鹏数学库开发指南”章节。

如**表2-8**所示为可替换的接口与MKL接口的映射关系。

版权所有 © 华为技术有限公司

表 2-8 FFT 的替换映射关系

MKL接口	KML对应接口	描述
DftiCreateDescriptor DftiSetValue DftiCommitDescriptor	kml_fft_plan_dft kml_fft_plan_dft_1d kml_fft_plan_dft_2d kml_fft_plan_dft_3d kml_fft_plan_dft_c2r kml_fft_plan_dft_c2r_1d kml_fft_plan_dft_c2r_2d kml_fft_plan_dft_c2r_3d kml_fft_plan_dft_r2c kml_fft_plan_dft_r2c_1d kml_fft_plan_dft_r2c_2d kml_fft_plan_dft_r2c_3d kml_fft_plan_guru64_dft kml_fft_plan_guru64_dft_c2r kml_fft_plan_guru64_dft_r2c kml_fft_plan_guru64_r2r kml_fft_plan_guru64_split_dft kml_fft_plan_guru64_split_dft_c2r kml_fft_plan_guru64_split_dft_r2c kml_fft_plan_guru_dft kml_fft_plan_guru_dft_c2r kml_fft_plan_guru_dft_r2c kml_fft_plan_guru_r2r kml_fft_plan_guru_split_dft kml_fft_plan_guru_split_dft_c2r kml_fft_plan_guru_split_dft_r2c kml_fft_plan_many_dft kml_fft_plan_many_dft_c2r	MKL构建FFT模型有三个步骤： 1. DftiCreateDescriptor接口创建一个问题描述。 2. 通过DftiSetValue接口设置对应的问题参数。 3. 最后通过DftiCommitDescriptor完成问题创建。 KML构建问题时根据问题类型选用不同的plan接口，不同的plan接口需要不同的问题参数配置，通过参数传递给plan接口，完成问题创建。在具体迁移过程中需要单独替换。 具体请参见**C语言迁移步骤**中给出的三个代码示例。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	描述
	kml_fft_plan_many_dft_r2c kml_fft_plan_many_r2r kml_fft_plan_r2r kml_fft_plan_r2r_1d kml_fft_plan_r2r_2d kml_fft_plan_r2r_3d kml_fft_plan_with_nthreads kml_fft_init_threads kml_fft_cleanup_threads	
DftiFreeDescriptor	kml_fft_destroy_plan kml_fft_free	此处为释放模型关联资源的对应接口。
DftiCopyDescriptor	无对应	-
DftiComputeForward	kml_fft_execute kml_fft_execute_dft kml_fft_execute_split_dft kml_fft_execute_dft_r2c kml_fft_execute_split_dft_r2c	MKL里面的C2C类问题、R2C类问题用DftiComputeForward，对应KML里面的c2c、r2c相关的执行函数。
DftiComputeBackward	kml_fft_execute kml_fft_execute_dft kml_fft_execute_split_dft kml_fft_execute_dft_c2r kml_fft_execute_split_dft_c2r	MKL里面的C2C类问题、C2R类问题用DftiComputeBackward，对应KML里面的c2c、c2r相关的执行函数。
DftiGetValue	无对应	-
DftiErrorClass	无对应	-
DftiErrorMessage	无对应	-

C 语言迁移步骤

- 例：C2C变换

  迁移前：

```
#include "mkl_cdft.h"
#include "mkl_cdft_types.h"

   DFTI_DESCRIPTOR_HANDLE desc_handle = NULL;
   MKL_LONG rank = 2;
   MKL_LONG length[2] = {2, 3};
   MKL_LONG istrides[3] = {0, 3, 1};
```

 版权所有 © 华为技术有限公司

```
    MKL_LONG ostrides[3] = {0, 3, 1};
    MKL_LONG nthreads = 1;
    MKL_LONG howmany = 2;
    MKL_LONG idistance = 2 * 3;
    MKL_LONG odistance = 2 * 3;
    double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16},
{-40, 8}, {-8, -8}};
    double *ri;
    ri = (double*)malloc(sizeof(double) * 12);
    double *ii;
    ii = (double*)malloc(sizeof(double) * 12);
    for (int i = 0; i < 12; i++) {
        ri[i] = init[i][0];
        ii[i] = init[i][1];
    }

    MKL_LONG s = DftiCreateDescriptor(&desc_handle, DFTI_DOUBLE, DFTI_COMPLEX, rank, length);
    s = DftiSetValue(desc_handle, DFTI_PLACEMENT, DFTI_NOT_INPLACE);
    s = DftiSetValue(desc_handle, DFTI_INPUT_STRIDES, istrides);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_STRIDES, ostrides);
    s = DftiSetValue(desc_handle, DFTI_THREAD_LIMIT, nthreads);
    s = DftiSetValue(desc_handle, DFTI_COMPLEX_STORAGE, DFTI_REAL_REAL);
    s = DftiSetValue(desc_handle, DFTI_NUMBER_OF_TRANSFORMS, howmany);
    s = DftiSetValue(desc_handle, DFTI_INPUT_DISTANCE, idistance);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_DISTANCE, odistance);
    s = DftiCommitDescriptor(desc_handle);
    double *ro;
    ro = (double*)malloc(sizeof(double) * 12);
    double *io;
    io = (double*)malloc(sizeof(double) * 12);
    DftiComputeForward(desc_handle, ri, ii, ro, io);
    DftiFreeDescriptor(&desc_handle);
    free(ri);
    free(ii);
    free(ro);
    free(io);
```

迁移后：

```
#include "kfft.h"

    int rank = 2;
    kml_fft_iodim64 *dims;
    dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
    dims[0].n = 2;
    dims[0].is = 3;
    dims[0].os = 3;
    dims[1].n = 3;
    dims[1].is = 1;
    dims[1].os = 1;
    int howmany_rank = 1;
    kml_fft_iodim64 *howmany_dims;
    howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 3;
    howmany_dims[0].os = 2 * 3;
    double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16},
{-40, 8}, {-8, -8}};
    double *ri;
    ri = (double*)kml_fft_malloc(sizeof(double) * 12);
    double *ii;
    ii = (double*)kml_fft_malloc(sizeof(double) * 12);
    for (int i = 0; i < 12; i++) {
        ri[i] = init[i][0];
        ii[i] = init[i][1];
    }
    double *ro;
    ro = (double*)kml_fft_malloc(sizeof(double) * 12);
    double *io;
    io = (double*)kml_fft_malloc(sizeof(double) * 12);
    kml_fft_plan plan;
```


版权所有 © 华为技术有限公司

```
    plan = kml_fft_plan_guru64_split_dft(rank, dims, howmany_rank, howmany_dims, ri, ii, ro, io,
KML_FFT_ESTIMATE);
    kml_fft_execute_split_dft(plan, ri, ii, ro, io);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(ri);
    kml_fft_free(ii);
    kml_fft_free(ro);
    kml_fft_free(io);

    /*
     * ro = {1.200000e+02, 1.338564e+02, 1.061436e+02, 1.360000e+02,
     *       1.120000e+02, 1.120000e+02, -8.000000e+01, 4.878461e+01,
     *       7.215390e+00, 1.600000e+01, -2.692820e+01, -1.307180e+01}
     */

    /*
     * io = {4.800000e+01, -1.385641e+01, 1.385641e+01, -3.200000e+01,
     *       -8.000000e+00, -8.000000e+00, -8.000000e+00, 7.846097e-01,
     *       -4.078461e+01, 2.400000e+01, -2.264102e+01, 4.664102e+01}
     */
```

- 例：R2C变换

  迁移前：

```
#include "mkl_cdft.h"
#include "mkl_cdft_types.h"

    DFTI_DESCRIPTOR_HANDLE desc_handle = NULL;
    MKL_LONG rank = 2;
    MKL_LONG length[2] = {2, 3};
    MKL_LONG istrides[3] = {0, 3, 1};
    MKL_LONG ostrides[3] = {0, 2, 1};
    MKL_LONG nthreads = 1;
    MKL_LONG howmany = 2;
    MKL_LONG idistance = 2 * 3;
    MKL_LONG odistance = 2 * 2;
    double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
    double *in;
    in = (double*)malloc(sizeof(double) * 12);
    for (int i = 0; i < 12; i++) {
        in[i] = init[i];
    }

    MKL_LONG s = DftiCreateDescriptor(&desc_handle, DFTI_DOUBLE, DFTI_REAL, rank, length);
    s = DftiSetValue(desc_handle, DFTI_PLACEMENT, DFTI_NOT_INPLACE);
    s = DftiSetValue(desc_handle, DFTI_INPUT_STRIDES, istrides);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_STRIDES, ostrides);
    s = DftiSetValue(desc_handle, DFTI_THREAD_LIMIT, nthreads);
    s = DftiSetValue(desc_handle, DFTI_COMPLEX_STORAGE, DFTI_COMPLEX_COMPLEX);
    s = DftiSetValue(desc_handle, DFTI_NUMBER_OF_TRANSFORMS, howmany);
    s = DftiSetValue(desc_handle, DFTI_INPUT_DISTANCE, idistance);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_DISTANCE, odistance);
    s = DftiSetValue(desc_handle, DFTI_CONJUGATE_EVEN_STORAGE, DFTI_COMPLEX_COMPLEX);
    s = DftiCommitDescriptor(desc_handle);

    MKL_Complex16 *out;
    out = (MKL_Complex16*)malloc(sizeof(MKL_Complex16) * 2 * 2 * howmany);
    DftiComputeForward(desc_handle, in, out);
    DftiFreeDescriptor(&desc_handle);
    free(in);
    free(out);
```

  迁移后：

```
#include "kfft.h"

    int rank = 2;
    kml_fft_iodim64 *dims;
    dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
```

 版权所有 © 华为技术有限公司

```
    dims[0].n = 2;
    dims[0].is = 3;
    dims[0].os = 2;
    dims[1].n = 3;
    dims[1].is = 1;
    dims[1].os = 1;
    int howmany_rank = 1;
    kml_fft_iodim64 *howmany_dims;
    howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 3;
    howmany_dims[0].os = 2 * 2;
    double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
    double *in;
    in = (double*)kml_fft_malloc(sizeof(double) * 12);
    for (int i = 0; i < 12; i++) {
        in[i] = init[i];
    }
    kml_fft_complex *out;
    out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
    kml_fft_plan plan;
    plan = kml_fft_plan_guru64_dft_r2c(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
    kml_fft_execute_dft_r2c(plan, in, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(in);
    kml_fft_free(out);

    /*
     * out = {{1.200000e+02, 0.000000e+00}, {1.200000e+02, -1.385641e+01},
     *        {1.360000e+02, 0.000000e+00}, {1.120000e+02, 0.000000e+00},
     *        {-8.000000e+01, 0.000000e+00}, {2.800000e+01, 2.078461e+01},
     *        {1.600000e+01, 0.000000e+00}, {-2.000000e+01, -3.464102e+01}}
     */
```

- 例：C2R变换

  迁移前：

```
#include "mkl_cdft.h"
#include "mkl_cdft_types.h"

    DFTI_DESCRIPTOR_HANDLE desc_handle = NULL;
    MKL_LONG rank = 2;
    MKL_LONG length[2] = {2, 3};
    MKL_LONG istrides[3] = {0, 2, 1};
    MKL_LONG ostrides[3] = {0, 3, 1};
    MKL_LONG nthreads = 1;
    MKL_LONG howmany = 2;
    MKL_LONG idistance = 2 * 2;
    MKL_LONG odistance = 2 * 3;
    double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
    MKL_Complex16 *in;
    in = (MKL_Complex16*)malloc(sizeof(MKL_Complex16) * 8);
    for (int i = 0; i < 8; i++) {
        in[i].real = init[i][0];
        in[i].imag = init[i][1];
    }

    MKL_LONG s = DftiCreateDescriptor(&desc_handle, DFTI_DOUBLE, DFTI_REAL, rank, length);
    s = DftiSetValue(desc_handle, DFTI_PLACEMENT, DFTI_NOT_INPLACE);
    s = DftiSetValue(desc_handle, DFTI_INPUT_STRIDES, istrides);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_STRIDES, ostrides);
    s = DftiSetValue(desc_handle, DFTI_THREAD_LIMIT, nthreads);
    s = DftiSetValue(desc_handle, DFTI_COMPLEX_STORAGE, DFTI_COMPLEX_COMPLEX);
    s = DftiSetValue(desc_handle, DFTI_NUMBER_OF_TRANSFORMS, howmany);
    s = DftiSetValue(desc_handle, DFTI_INPUT_DISTANCE, idistance);
    s = DftiSetValue(desc_handle, DFTI_OUTPUT_DISTANCE, odistance);
```

 版权所有 © 华为技术有限公司

```
    s = DftiSetValue(desc_handle, DFTI_CONJUGATE_EVEN_STORAGE, DFTI_COMPLEX_COMPLEX);
    s = DftiCommitDescriptor(desc_handle);

    double *out;
    out = (double*)malloc(sizeof(double) * 12);
    DftiComputeBackward(desc_handle, in, out);
    DftiFreeDescriptor(&desc_handle);
    free(in);
    free(out);
```

迁移后：

```
#include "kfft.h"

    int rank = 2;
    kml_fft_iodim64 *dims;
    dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
    dims[0].n = 2;
    dims[0].is = 2;
    dims[0].os = 3;
    dims[1].n = 3;
    dims[1].is = 1;
    dims[1].os = 1;
    int howmany_rank = 1;
    kml_fft_iodim64 *howmany_dims;
    howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 2;
    howmany_dims[0].os = 2 * 3;
    double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
    kml_fft_complex *in;
    in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
    for (int i = 0; i < 8; i++) {
        in[i].r = init[i][0];
        in[i].i = init[i][1];
    }
    double *out;
    out = (double*)kml_fft_malloc(sizeof(double) * 12);
    kml_fft_plan plan;
    plan = kml_fft_plan_guru64_dft_c2r(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
    kml_fft_execute_dft_c2r(plan, in, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(in);
    kml_fft_free(out);

    /*
     * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
     *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
     *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
     */
```

须知

由于软件架构原因，在业务代码中若有多次计算不同类型的傅里叶变换，则需要构建对应的不同Plan，并且与MKL类似，同类型的变换构建一次Plan可以多次使用。

Fortran 接口迁移步骤

Fortran接口迁移与**C语言迁移步骤**一致。

版权所有 © 华为技术有限公司

2.6 线性代数运算库（LAPACK）迁移

可替换性

KML_LAPACK对外提供的接口，与MKL_LAPACK的接口参数和函数名完全一致，可以直接替换库使用。

如**表1 LAPACK的替换映射关系**所示为可替换的接口与MKL中LAPACK的Fortran接口的映射关系（仅给出已优化的接口）。

表 2-9 LAPACK 的替换映射关系

MKL接口	KML对应接口	数据类型	描述
?getrf	?getrf	s、d、c、z	计算矩阵A的LU分解
?geqrf	?geqrf	s、d、c、z	计算矩阵的QR分解
?gerqf	?gerqf	s、d、c、z	计算矩阵的RQ分解
?geqlf	?geqlf	s、d、c、z	计算矩阵的QL分解
?gelqf	?gelqf	s、d、c、z	计算矩阵的LQ分解
?ppsv	?ppsv	s、d、c、z	计算矩阵A的Cholesky分解，并通过分解结果求解线性方程组
?ptsv	?ptsv	s、d、c、z	求解线性方程组，其中系数矩阵A为（共轭）对称正定三对角矩阵。
?getri	?getri	s、d、c、z	根据?getrf得到的LU分解结果计算逆矩阵
?syevd	?syevd	s、d	计算对称（Hermite）矩阵的全部特征值和特征向量
?heevd	?heevd	c、z	计算对称（Hermite）矩阵的全部特征值和特征向量
?sytrd	?sytrd	s、d	将对称矩阵矩阵通过相似变换成对称三对角矩阵
?hetrd	?hetrd	c、z	将Hermite矩阵通过相似变换成对称三对角矩阵
?potrf	?potrf	s、d、c、z	计算对称正定矩阵或者Hermite正定矩阵的Cholesky分解
?pttrf	?pttrf	s、d、c、z	计算实（共轭）对称正定三对角矩阵A的LDL^*或U^*DU分解

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
?potri	?potri	s、d、c、z	计算对称正定矩阵的逆。
?gesv	?gesv	s、d、c、z	求解线性方程组$A * X = B$，其中A是$n \times n$的矩阵，X和B为$n \times nrhs$的矩阵。
?orglq	?orglq	s、d	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行
?unglq	?unglq	c、z	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行
?ormlq	?ormlq	s、d	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gelqf计算得到。
?unmlq	?unmlq	c、z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gelqf计算得到。
?orgql	?orgql	s、d	生成具有正交列的实/复矩阵Q，且该矩阵定义为K个M阶基本反射器的乘积的前N列
?ungql	?ungql	c、z	生成具有正交列的实/复矩阵Q，且该矩阵定义为K个M阶基本反射器的乘积的前N列
?ormql	?ormql	s、d	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqlf计算得到。
?unmql	?unmql	c、z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqlf计算得到。
?orgqr	?orgqr	s、d	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
?ungqr	?ungqr	c、z	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行
?orgrq	?orgrq	s、d	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行
?ungrq	?ungrq	c、z	生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行
?ormrq	?ormrq	s、d	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gerqf计算得到。
?unmrq	?unmrq	c、z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gerqf计算得到。
?ormqr	?ormqr	s、d	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqrf计算得到。
?unmqr	?unmqr	c、z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqrf计算得到。
?syev	?syev	s、d	计算对称（Hermite）矩阵的所有特征值与特征向量。
?heev	?heev	c、z	计算对称（Hermite）矩阵的所有特征值与特征向量。
?pttrs	?pttrs	s、d、c、z	求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。
?ptts2	?ptts2	s、d、c、z	求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。
?lasr	?lasr	s、d、c、z	对矩阵A做平面旋转操作。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
?gtsv	?gtsv	s、d、c、z	求解线性方程组A*X=B，其中系数矩阵A为一般三对角矩阵。
?gttrf	?gttrf	s、d、c、z	计算一般三对角矩阵A的LU分解。
?gttrs	?gttrs	s、d、c、z	求解三对角方程A*X=B或A**T *X=B或A**H *X=B，其中系数矩阵A由?gttrf分解而来。
?sytrd_2stage	?sytrd_2stage	s、d、c、z	将对称矩阵或Hermite矩阵A转为对称或Hermite三对角矩阵T，
?trtrs	?trtrs	s、d、c、z	求解三角方程A * X = B，或A**T * X = B。
?laset	?laset	s、d、c、z	初始化m*n的矩阵，且将对角线元素设置为beta，非对角线元素设置为alpha。
?sptrf	?sptrf	s、d、c、z	计算压缩对称矩阵的LDL*或U*DU分解。
?hptrf	?hptrf	s、d、c、z	计算压缩Hermite矩阵的LDL*或U*DU分解。
?pptrf	?pptrf	s、d、c、z	计算压缩存储的对称正定矩阵的LLT或UTU分解。
?pptrs	?pptrs	s、d、c、z	求解三对角方程A*X=B或A**T *X=B或A**H *X=B。其中系数矩阵A由?gttrf分解而来。
?pptri	?pptri	s、d、c、z	计算压缩存储的对称正定矩阵的逆矩阵。
?getrs	?getrs	s、d、c、z	求解一般线性方程A*X=B或A**T * X=B或A**H * X=B。其中系数矩阵A由?getrf分解而来。
?posv	?posv	s、d、c、z	求解线性方程组$AX = B$。其中A为实对称或共轭对称的$n \times n$正定矩阵；X和B为$n \times nrhs$矩阵。
?trtri	?trtri	s、d、c、z	计算上/下三角矩阵的逆矩阵。

版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
?laswp	?laswp	s、d、c、z	对矩阵做一系列的行交换操作。
?lascl	?lascl	s、d、c、z	对矩阵做scalar操作。
?lange	?lange	s、d、c、z	计算矩阵的范数，包含：1范数，F范数，无穷范数等。
?sgesv	?gesv	ds、zc	求解线性方程组$A * X = B$，其中A是$n \times n$的矩阵，X和B为$n \times nrhs$的矩阵。
?gels	?gels	s、d、c、z	使用矩阵的QR或LQ分解来求解超定或欠定线性方程组。
?gelsd	?gelsd	s、d、c、z	使用分治法对线性最小二乘问题求解其最小范数解。
?gelss	?gelss	s、d、c、z	使用SVD方法对线性最小二乘问题求解其最小范数解。
?steqr	?steqr	s、d、c、z	使用QL或QR方法求解对称三对角矩阵的特征值和特征向量。
?gtts2	?gtts2	s、d、c、z	求解三对角方程A*X=B或A**T *X=B或A**H *X=B，其中系数矩阵A由?gttrf分解而来。
?ormbr	?ormbr	s、d	计算C=Q*C或Q**T*C或C*Q或C*Q**T或C=P*C或P**T*C或C*P或C*P**T，
?unmbr	?unmbr	c、z	计算C=Q*C或Q**T*C或C*Q或C*Q**T或C=P*C或P**T*C或C*P或C*P**T，
?orgtr	?orgtr	s、d	通过反射因子生成正交矩阵Q，其中反射因子由SYTRD计算得来。
?ungtr	?ungtr	c、z	通过反射因子生成正交矩阵Q，其中反射因子由SYTRD计算得来。
?lacpy	?lacpy	s、d、c、z	拷贝矩阵A的所有元素或部分元素到矩阵B。

 版权所有 © 华为技术有限公司

MKL接口	KML对应接口	数据类型	描述
?stedc	?stedc	s、d、c、z	对称三对角矩阵的特征值求解，使用Divide-and-Conquer算法。
?gesvd	?gesvd	s、d、c、z	一般矩阵SVD分解。
?gebrd	?gebrd	s、d、c、z	一般矩阵变换到双对角矩阵。
?bdsdc	?bdsdc	s、d	计算N*N的上/下双对角矩阵B的奇异值分解，使用分治算法。
?gesdd	?gesdd	s、d、c、z	计算矩形矩阵的奇异值分解，可以用来计算左和右奇异向量，使用分治算法。
?bdsqr	?bdsqr	s、d、c、z	使用对双对角矩阵的QR算法的SVD分解来求解奇异值或左右奇异向量。

以上接口均为Fortran接口，在C语言中使用时需在接口名称后增加下划线，且参数类型均为指针类型，请参见《Kunpeng HPCKit 26.1.RC1 开发指南》中“鲲鹏数学库 开发指南 > 应用数学库 > KML_LAPACK库函数说明 > 函数定义”章节。

若需使用LAPACKE接口（MKL中标准的C接口），需在编译开源Netlib LAPACK时增加对LAPACKE封装库的编译，请参见C语言迁移步骤中说明。

C 语言迁移步骤

步骤1 MKL LAPACK提供Fortran和LAPACK（C语言封装）两种接口，KML LAPACK提供的Fortran接口与MKL完全一致，C语言调用时将头文件#include "mkl.h"替换为#include "klapack.h" 。

----结束

例如：

迁移前使用Fortran接口：

```
/* Declare the dgetrf interface */
void dgetrf_(int *m, int *n, double *a, int *lda, int *ipiv, int *info);
......
    int m = 4;
    int n = 4;
    int lda = 4;
    int ipiv[4];
    int info = 0;
    double a[] = {1.80,  5.25,  1.58, -1.11,
                  2.88, -2.95, -2.69, -0.66,
                  2.05, -0.95, -2.90, -0.59,
                  -0.89, -3.80, -1.04,  0.80};
    dgetrf_(&m, &n, a, &lda, ipiv, &info);
```

迁移前使用LAPACKE接口：

 版权所有 © 华为技术有限公司

```
#include "mkl.h"
......
    int m = 4;
    int n = 4;
    int lda = 4;
    int ipiv[4];
    double a[] = {1.80,  5.25,  1.58, -1.11,
                  2.88, -2.95, -2.69, -0.66,
                  2.05, -0.95, -2.90, -0.59,
                  -0.89, -3.80, -1.04,  0.80};
    int info = LAPACKE_dgetrf(LAPACK_COL_MAJOR, m, n, a, lda, ipiv);
```

迁移后使用KML LAPACK的Fortran接口：

```
#include "klapack.h"
......
    int m = 4;
    int n = 4;
    int lda = 4;
    int ipiv[4];
    int info = 0;
    double a[] = {1.80,  5.25,  1.58, -1.11,
                  2.88, -2.95, -2.69, -0.66,
                  2.05, -0.95, -2.90, -0.59,
                  -0.89, -3.80, -1.04,  0.80};
    dgetrf_(&m, &n, a, &lda, ipiv, &info);
```

与使用MKL的Fortran接口一致，包含klapack.h可得到Fortran接口的C函数声明。

Fortran 接口迁移步骤

Fortran接口与MKL一致，代码无需修改。

2.7 迭代求解器（ITERATIVE SOLVER）迁移

可替换性

KML_SOLVER对外提供的接口，与MKL的接口参数和函数名不一致，需要修改源码。

如**表1 SOLVER的替换映射关系**所示为可替换的接口与MKL中接口的映射关系。

表 2-10 SOLVER 的替换映射关系

MKL接口	KML接口
dcg_init	KmlIssCgInit?I
dcg	KmlIssCgSolve?I
dcg_get	KmlIssCgGet?I?
mkl_sparse_destroy	KmlIssCgClean?I

以上接口均为C接口，请参见**《Kunpeng HPCKit 26.1.RC1 开发指南》**中“鲲鹏数学库 开发指南 > 应用数学库 > KML_SOLVER库函数说明 > 函数定义（单机版）”章节。

版权所有 © 华为技术有限公司

C 语言迁移步骤

MKL提供了C接口，KML_SOLVER同样提供了C接口，但接口名及接收参数不一致，需要进行对应功能的切换。

1. 初始化。

   迁移前：

   ```
   const MKL_INT n = 8;
   MKL_INT ia[9];
   MKL_INT ja[18];
   double a[18];
   MKL_INT rci_request;
   double rhs[8] = {0.0};
   double solution[8] = {0.0};
   double expected_sol[8];
   MKL_INT ipar[128];
   double dpar[128], tmp[4 * 8] = {0.0};
   sparse_matrix_t       csrA;
   struct matrix_descr descrA;
   sparse_operation_t    transA;
   ......
   mkl_sparse_d_create_csr ( &csrA, SPARSE_INDEX_BASE_ONE, n, n, ia, ia+1, ja, a );
   mkl_sparse_d_mv( transA, 1.0, csrA, descrA, expected_sol, 0.0, rhs);
   dcg_init (&n, solution, rhs, &rci_request, ipar, dpar, tmp);
   ```

   迁移后：

   ```
   int n = 8;
   int ia[9];
   int ja[18];
   double a[18];
   void* handle;
   int error = KmlIssCgInitDI(&handle, &n, a, ja, ia);
   ```

2. 求解。

   迁移前：

   ```
   dcg (&n, solution, rhs, &rci_request, ipar, dpar, tmp);
   ```

   迁移后：

   ```
   double b[8];
   /* Solution vector */
   double x[8];
   int nrhs = 1;/* Number of right-hand sides */
   int ldx=n, ldb=n;/*!Leading dimension of B and X */
   int error = KmlIssCgSolveDI(&handle, &nrhs, x, &ldx, b, &ldb);
   ```

3. 获取迭代参数。

   迁移前：

   ```
   MKL_INT itercount
   dcg_get (&n, solution, rhs, &rci_request, ipar, dpar, tmp, &itercount);
   ```

   迁移后：

   ```
   int *data;  // user data.
   int nd = 1;
   int error = KmlIssCgGetDII(&handle, ITERATION_COUNT, data, nd);
   ```

4. 清除。

   迁移前：

   ```
   mkl_sparse_destroy(csrA);
   ```

   迁移后：

   ```
   KmlIssCgCleanDI(&handle);
   ```

5. 头文件。

   迁移前：

   ```
   #include "mkl_rci.h"
   #include "mkl_spblas.h"
   ```


版权所有 © 华为技术有限公司

#include "mkl_service.h"

迁移后：

#include "kml_iss.h"

6. 编译链接库。

   替换MKL相关的链接选项，具体选项请参见《Kunpeng HPCKit 26.1.RC1 安装指南》安装KML。

2.8 直接求解器（DIRECT SOLVER）迁移

可替换性

MKL中包含单机版直接求解器PARDISO（Parallel Direct Sparse Solver Interface）和集群版直接求解器PARDISO-Cluster（Parallel Direct Sparse Solver for Cluster Interface）。前者为shared-memory版本的并行稀疏求解器，而后者则和SCADSS一样，支持MPI和OPENMP技术的混合模式。直接求解器包含Analyze、Factorize和Solve三个核心阶段，每个阶段都可以单独调用。这一点与用户的实际问题相关，比如用户在求解部分包含时间项的偏微分方程的时候，稀疏矩阵是保持不变的，因此Analyze和Factorize只需要进行一次，重复调用Solve阶段即可。再比如，稀疏矩阵的数值发生了变化，但是矩阵的非零元位置未发生变化，那么Analyze只需要进行一次，重复调用Solve阶段即可。在这里给出MKL中的direct solver与SCADSS的接口映射关系。

表 2-11 PARDISO-Cluster 与 SCADSS 的替换映射关系

PARDISO-Cluster接口	SCADSS接口

mtype=2;或者mtype = 4;
phase = 11;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &comm, &error);
	KmlScadssSpdAnalyzeDI(&handle);或者
	KmlScadssHpdAnalyzeZI(&handle);

mtype=2;或者mtype = 4;
phase = 22;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &comm, &error);
	KmlScadssSpdFactorizeDI(&handle);或者
	KmlScadssHpdFactorizeZI(&handle);

mtype=2;或者mtype = 4;
phase = 33;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &comm, &error);
	KmlScadssSpdSolveDI(&handle);或者
	KmlScadssHpdSolveZI(&handle);

mtype=2;或者mtype = 4;
phase = -1;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &comm, &error);
	KmlScadssSpdCleanDI(&handle);或者
	KmlScadssHpdCleanZI(&handle);

版权所有 © 华为技术有限公司

说明

1. MKL中direct solver通过mtype参数确定输入稀疏矩阵的类型，mtype=2表示SPD矩阵，mtype=4则表示HPD矩阵。
2. phase参数控制MKL direct solver执行的操作，除上表中列举的取值外，还可以取别的值。例如取12时表示执行Analyze，Factorize两个过程。

表 2-12 PARDISO 与 SCADSS 的替换映射关系

PARDISO接口	SCADSS接口

mtype=2;或者mtype = 4;
phase = 11;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &error);
	KmlScadssSpdAnalyzeDI(&handle);或者
	KmlScadssHpdAnalyzeZI(&handle);

mtype=2;或者mtype = 4;
phase = 22;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &error);
	KmlScadssSpdFactorizeDI(&handle);或者
	KmlScadssHpdFactorizeZI(&handle);

mtype=2;或者mtype = 4;
phase = 33;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &error);
	KmlScadssSpdSolveDI(&handle);或者
	KmlScadssHpdSolveZI(&handle);

mtype=2;或者mtype = 4;
phase = -1;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x, &error);
	KmlScadssSpdCleanDI(&handle);或者
	KmlScadssHpdCleanZI(&handle);

C 语言迁移步骤

1. **迁移前（调用PARDISO）。**

```
int n = 8;
int nnz = 17;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};

int nrhs = 1;     // Number of right hand sides.
double b[8]={4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
```


版权所有 © 华为技术有限公司

```
double x[8];

int mtype = 2;
void *pt[64];
int iparm[64];
int maxfct, mnum, phase, error, msglvl, perm;
pardisoinit(pt, &mtype, iparm);
iparm[34] = 1;
maxfct = 1;       // Maximum number of numerical factorizations.
mnum = 1;         // Which factorization to use.
msglvl = 0;       // Print statistical information in file.
error = 0;        // Initialize error flag.

phase = 11;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x,
&error); // Analyze
printf ("Reordering completed ... ");

phase = 22;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x,
&error); // Factorize
printf ("\nFactorization completed ... ");

phase = 33;
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x,
&error); // Solve
printf("\nSolve completed ... ");

printf("\nThe solution of the system is: ");
for(int i = 0; i < n; i++){
    printf ("\n x [%d] = % f", i, x[i]);
}
printf("\n");

phase = -1;      // Release internal memory.
pardiso(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm, &msglvl, b, x,
&error);
```

迁移前（调用PARDISO-Cluster）。

```
int n = 8;
int nnz = 17;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};

/* RHS and solution vectors. */
int nrhs = 1;     /* Number of right hand sides. */
double b[8]={4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];

int mtype = 2;
void *pt[64];
int iparm[64];
int maxfct, mnum, phase, error, msglvl, perm;
for (int i = 0; i < 64; i++) {
    iparm[i] = 0;
    pt[i] = 0;
}
iparm[0] = 1;
iparm[1] = 2;
iparm[17] = -1;
iparm[18] = -1;
iparm[34] = 1;
maxfct = 1;       // Maximum number of numerical factorizations.
mnum = 1;         // Which factorization to use.
msglvl = 0;       // Print statistical information in file.
error = 0;        // Initialize error flag.

MPI_Init(NULL, NULL);
int size, rank;
```


版权所有 © 华为技术有限公司

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Fint comm = MPI_Comm_c2f(MPI_COMM_WORLD);

phase = 11;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm,
&msglvl, b, x, &comm, &error); // Analyze

phase = 22;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm,
&msglvl, b, x, &comm, &error); // Factorize

phase = 33;
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm,
&msglvl, b, x, &comm, &error); // Solve

if (rank == 0){
    printf("The solution of the system is: ");
    for(int i = 0; i < n; i++){
        printf ("\n x [%d] = %f", i, x[i]);
    }
    printf("\n");
}

phase = -1;      // Release internal memory.
cluster_sparse_solver(pt, &maxfct, &mnum, &mtype, &phase, &n, a, ia, ja, &perm, &nrhs, iparm,
&msglvl, b, x, &comm, &error);

MPI_Finalize();
```

迁移后：

```
int n = 8;
int nrhs = 1;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};

MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

KmlSolverMatrixStore storeA;
storeA.indexType = KMLSS_INDEX_INT32;
storeA.valueType = KMLSS_VALUE_FP64;
storeA.format = KMLSS_MATRIX_STORE_CSR;
if (rank == 0) {
    storeA.nRow = n;
    storeA.nCol = n;
    storeA.csr.rowOffset = ia;
    storeA.csr.colIndex = ja;
    storeA.csr.value = a;
} else {
    storeA.nRow = 0;
    storeA.nCol = 0;
    storeA.csr.rowOffset = nullptr;
    storeA.csr.colIndex = nullptr;
    storeA.csr.value = nullptr;
}

KmlSolverMatrixOption optA;
optA.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optA.type = KMLSS_MATRIX_GEN;

KmlScasolverMatrixOption scaOptA;
if (rank == 0) {
    scaOptA.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                        KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                        KMLSS_MATRIX_OPTIONS_PARTITION;
    scaOptA.partition.type = KMLSS_MATRIX_PARTITION_ROW;
```


版权所有 © 华为技术有限公司

```
    scaOptA.globalNumRows = n;
    scaOptA.globalNumCols = n;
    scaOptA.partition.localBegin = 0;
} else {
    scaOptA.fieldMask = 0;
}

KmlScasolverMatrix *A;
ierr = KmlScasolverMatrixCreate(&A, &storeA, &optA, &scaOptA);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create A: %d\n", ierr);
    return 1;
}

// Create vector b
double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
KmlSolverMatrixStore storeB;
storeB.indexType = KMLSS_INDEX_INT32;
storeB.valueType = KMLSS_VALUE_FP64;
storeB.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
if (rank == 0) {
    storeB.nRow = n;
    storeB.nCol = nrhs;
    storeB.dense.value = b;
    storeB.dense.ld = n;
} else {
    storeB.nRow = 0;
    storeB.nCol = 0;
    storeB.dense.value = nullptr;
    storeB.dense.ld = 0;
}

KmlSolverMatrixOption optB;
optB.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optB.type = KMLSS_MATRIX_GEN;

KmlScasolverMatrixOption scaOptB;
if (rank == 0) {
    scaOptB.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                        KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                        KMLSS_MATRIX_OPTIONS_PARTITION;
    scaOptB.partition.type = KMLSS_MATRIX_PARTITION_ROW;
    scaOptB.partition.localBegin = 0;
    scaOptB.globalNumRows = n;
    scaOptB.globalNumCols = nrhs;
} else {
    scaOptB.fieldMask = 0;
}

KmlScasolverMatrix *B;
ierr = KmlScasolverMatrixCreate(&B, &storeB, &optB, &scaOptB);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create b: %d\n", ierr);
    return 1;
}

// Create vector x
double x[8] = {0};
KmlSolverMatrixStore storeX;
storeX.indexType = KMLSS_INDEX_INT32;
storeX.valueType = KMLSS_VALUE_FP64;
storeX.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
if (rank == 0) {
    storeX.nRow = n;
    storeX.nCol = nrhs;
    storeX.dense.value = x;
    storeX.dense.ld = n;
} else {
    storeX.nRow = 0;
```

 版权所有 © 华为技术有限公司

```
    storeX.nCol = 0;
    storeX.dense.value = nullptr;
    storeX.dense.ld = 0;
}

KmlSolverMatrixOption optX;
optX.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optX.type = KMLSS_MATRIX_GEN;

KmlScasolverMatrixOption scaOptX;
if (rank == 0) {
    scaOptX.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                        KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                        KMLSS_MATRIX_OPTIONS_PARTITION;
    scaOptX.partition.type = KMLSS_MATRIX_PARTITION_ROW;
    scaOptX.partition.localBegin = 0;
    scaOptX.globalNumRows = n;
    scaOptX.globalNumCols = nrhs;
} else {
    scaOptX.fieldMask = 0;
}

KmlScasolverMatrix *X;
ierr = KmlScasolverMatrixCreate(&X, &storeX, &optX, &scaOptX);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create x: %d\n", ierr);
    return 1;
}

// Init solver
KmlDssInitOption opt;
opt.fieldMask = KMLDSS_INIT_OPTION_BWR_MODE | KMLDSS_INIT_OPTION_NTHREADS;
opt.bwrMode = KMLDSS_BWR_OFF;
opt.nThreads = 32;

KmlScadssInitOption scaOpt;
scaOpt.fieldMask = KMLSCADSS_OPTIONS_COMM;
scaOpt.comm = comm;

KmlScadssSolver *solver;
ierr = KmlScadssInit(&solver, &opt, &scaOpt);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssInit: %d\n", ierr);
    return ierr;
}

// Analyze
KmlDssAnalyzeOption optAnalyze;
optAnalyze.fieldMask = KMLDSS_ANALYZE_OPTION_MATCHING_TYPE |
KMLDSS_ANALYZE_OPTION_RDR_TYPE |
                       KMLDSS_ANALYZE_OPTION_NTHREADS_RDR;
optAnalyze.matchingType = KMLDSS_MATCHING_OFF;
optAnalyze.rdrType = KMLDSS_RDR_KRDR;
optAnalyze.nThreadsRdr = 1;

KmlScadssAnalyzeOption scaOptAnalyze;
scaOptAnalyze.fieldMask = 0;

ierr = KmlScadssAnalyze(solver, A, &optAnalyze, &scaOptAnalyze);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssAnalyze: %d\n", ierr);
    return ierr;
}

// Factorize
KmlDssFactorizeOption optFact;
optFact.fieldMask = KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD;
optFact.perturbationThreshold = 1e-8;
```

 版权所有 © 华为技术有限公司

```
KmlScadssFactorizeOption scaOptFact;
scaOptFact.fieldMask = 0;

ierr = KmlScadssFactorize(solver, A, &optFact, &scaOptFact);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssFactorize: %d\n", ierr);
    return ierr;
}

// Solve
KmlDssSolveOption optSolve;
optSolve.fieldMask = KMLDSS_SOLVE_OPTION_SOLVE_STAGE |
KMLDSS_SOLVE_OPTION_REFINE_METHOD;
optSolve.stage = KMLDSS_SOLVE_ALL;
optSolve.refineMethod = KMLDSS_REFINE_OFF;

KmlScadssSolveOption scaOptSolve;
scaOptSolve.fieldMask = 0;

ierr = KmlScadssSolve(solver, B, X, &optSolve, &scaOptSolve);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssSolve: %d\n", ierr);
    return ierr;
}

// Output result x
if (rank == 0) {
    printf("Result of first factorize and solve:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf ", x[i]);
    }
    printf("\n");
}
MPI_Finalize();
```

2. 头文件

   迁移前（PARDISO）：

   #include "mkl_types.h"

   #include "mkl_cluster_sparse_solver.h"

   迁移前（PARDISO-Cluster）：

   #include "mkl_pardiso.h"

   #include "mkl_types.h"

   迁移后：

   #include "kml_scadss.h"

3. 编译链接库。

   替换MKL相关的链接选项， 具体选项请参见**《Kunpeng HPCKit 26.1.RC1 安装指南》**安装KML。

2.9 线性代数运算库（SCALAPACK）迁移

可替换性

如**表SCALAPACK的替换映射关系**所示为可替换的接口与MKL中SCALAPACK的Fortran接口的映射关系（仅给出已优化的接口）。

版权所有 © 华为技术有限公司

表 2-13 SCALAPACK 的替换映射关系

MKL接口	KML接口	数据类型	描述
p?getrf	p?getrf	s、d、c、z	计算矩阵A的LU分解，允许行交换（ partial pivoting ）。
p?potrf	p?potrf	s、d、c、z	计算对称正定矩阵或者Hermite正定矩阵的Cholesky分解。
p?geqrf	p?geqrf	s、d、c、z	计算矩阵的QR分解，即 $A = Q \times R$。
p?gels	p?gels	s、d、c、z	使用A的QR或LQ分解求解满秩矩阵A或其转置的超定或欠定实线性系统。

C 语言迁移步骤

步骤1 MKL SCALAPACK提供Fortran和SCALAPACK（ C语言封装 ）两种接口，KML SCALAPACK提供的Fortran接口与MKL完全一致，C语言调用时将头文件#include "mkl.h"替换为#include "kscalapack.h" 。

----结束

Fortran 接口迁移步骤

Fortran接口与MKL一致，代码无需修改。

2.10 短向量数学库（ SVML ）迁移

可替换性

Intel SVML（ Intel Short Vector Math Library，英特尔短向量数学库 ）是英特尔提供给编译器使用的自动向量化接口，可以将一次处理一对标量运算转换为一次并行处理多对向量运算，属于编译器对标量运算的一种优化，Intel SVML的接口并不对用户提供。KML_SVML提供了三十个接口供编译器调用，如**表1 编译器向量化接口**所示。

表 2-14 编译器向量化接口

SVML接口	MATH接口	描述
__svml_sin? __svml_sinf?	_ZGVnN2v_sin _ZGVnN4v_sinf	正弦函数
__svml_cos? __svml_cosf?	_ZGVnN2v_cos _ZGVnN4v_cosf	余弦函数
__svml_tan? __svml_tanf?	_ZGVnN2v_tan _ZGVnN4v_tanf	正切函数

版权所有 © 华为技术有限公司

SVML接口	MATH接口	描述
__svml_atan? __svml_atanf?	_ZGVnN2v_atan _ZGVnN4v_atanf	反正切函数
__svml_atan2? __svml_atan2f?	_ZGVnN2vv_atan2 _ZGVnN4vv_atan2f	src1/src2的反正切值
__svml_sinh? __svml_sinhf?	_ZGVnN2v_sinh _ZGVnN4v_sinhf	双曲正弦函数
__svml_cosh? __svml_coshf?	_ZGVnN2v_cosh _ZGVnN4v_coshf	双曲余弦函数
__svml_tanh? __svml_tanhf?	_ZGVnN2v_tanh _ZGVnN4v_tanhf	双曲正切函数
__svml_exp? __svml_expf?	_ZGVnN2v_exp _ZGVnN4v_expf	指数函数（base e）
__svml_exp2? __svml_exp2f?	_ZGVnN2v_exp2 _ZGVnN4v_exp2f	指数函数（base 2）
__svml_expm1? __svml_expm1f?	_ZGVnN2v_expm1 _ZGVnN4v_expm1f	指数值减1（base e）
__svml_log? __svml_logf?	_ZGVnN2v_log _ZGVnN4v_logf	对数函数（base e）
__svml_log10? __svml_log10f?	_ZGVnN2v_log10 _ZGVnN4v_log10f	对数函数（base 10）
__svml_log1p? __svml_log1pf?	_ZGVnN2v_log1p _ZGVnN4v_log1pf	(1+x)的对数函数（base e）
__svml_pow? __svml_powf?	_ZGVnN2vv_pow _ZGVnN4vv_powf	幂函数

使用 GCC 编译

GCC自动向量化调用方法请参见《Kunpeng HPCKit 26.1.RC1 开发指南》中“鲲鹏数学库 开发指南 > 基础数学库 > KML_SVML库函数说明 > 编译器自动向量化 > GCC向量化方法 ”章节。

使用毕昇编译器编译

毕昇编译器自动向量化调用方法请参见《Kunpeng HPCKit 26.1.RC1 开发指南》中“鲲鹏数学库开发指南 > 基础数学库 > KML_SVML库函数说明 > 编译器自动向量化 > 毕昇编译器向量化方法”章节。

 版权所有 © 华为技术有限公司

3 其他常用数学库迁移到 KML

3.1 LIBM、MATHLIB、LIBIMF 迁移到 KML_MATH

可替换性

表 3-1 LIBM、MATHLIB、LIBIMF 的替换映射关系

接口名	KML接口名	数据类型	描述
sin	sin	double、float、long double	正弦函数
cos	cos	double、float、long double	余弦函数
sincos	sincos	double、float、long double	正弦、余弦函数
tan	tan	double、float、long double	正切函数
asin	asin	double、float	反正弦函数
asind	asind	double、float	反正弦函数（入参为角度）
acos	acos	double、float	反余弦函数
atan	atan	double、float	反正切函数
atan2	atan2	double、float、long double	反正切函数
cosd	cosd	double、float、long double	余弦函数（入参为角度）
sinh	sinh	double、float	双曲正弦函数
cosh	cosh	double、float	双曲余弦函数

 版权所有 © 华为技术有限公司

接口名	KML接口名	数据类型	描述
tanh	tanh	double、float	双曲正切函数
asinh	asinh	double、float	反双曲正弦函数
acosh	acosh	double、float	反双曲余弦函数
atanh	atanh	double、float	反双曲正切函数
exp	exp	double、float、long double	指数函数（base e）
exp2	exp2	double、float	指数函数（base 2）
log	log	double、float、long double	对数函数（base e）
log2	log2	double、float	对数函数（base 2）
log10	log10	double、float	对数函数（base 10）
pow	pow	double、float、long double	幂函数
sqrt	sqrt	double、float、long double	平方根函数
cbrt	cbrt	double、float	立方根函数
fmod	fmod	double、float	浮点数求模
tgamma	tgamma	double、float	伽马函数
__powr8i4	powr8i4	-	80位扩展精度幂函数

高性能版本 C 语言迁移步骤

步骤1 识别源码中是否包含开源基础数学库的头文件：<mathimf.h>/<math.h> / <mathlib.h>。

步骤2 将开源头文件替换成<km.h>。

步骤3 在编译选项中将-lm或-lmathlib替换为-lkm -lm。

步骤4 添加环境变量，/PATH_TO_KM为安装路径。

```
export LD_LIBRARY_PATH=/PATH_TO_KM:$LD_LIBRARY_PATH
```

----结束

高精度版本 C 语言迁移步骤

步骤1 识别源码中是否包含开源基础数学库的头文件：<mathimf.h>/<math.h>。

步骤2 将开源头文件替换成<km.h> 。

步骤3 在编译选项中将-lm -fp-model=precise替换为-lkm_l9 -lm。

 版权所有 © 华为技术有限公司

步骤4 添加环境变量，/PATH_TO_KM为安装路径。

```
export LD_LIBRARY_PATH=/PATH_TO_KM:$LD_LIBRARY_PATH
```

----结束

Fortran 接口迁移步骤

Fortran接口迁移与**高性能版本C语言迁移步骤**一致。

3.2 OpenBLAS 迁移到 KML_BLAS

可替换性

KML_BLAS与OpenBLAS接口定义完全相同。

C 语言迁移步骤

OpenBLAS包括了开源的netlib LAPACK，如只用到了BLAS部分的接口可参照以下步骤迁移。

步骤1 先将识别出使用到OpenBLAS的源码调用的头文件#include <cblas.h> 替换成#include <kblas.h> 。

步骤2 在编译选项中将-lopenblas替换为-lkblas。

步骤3 添加环境变量，/PATH_TO_KBLAS为安装路径。

```
export LD_LIBRARY_PATH=/PATH_TO_KBLAS:$LD_LIBRARY_PATH
```

----结束

若遇到同时使用BLAS部分与LAPACK部分的接口，请参见以下步骤进行迁移。

步骤1 在编译选项中将-lopenblas替换为-lkblas -lklapack。

步骤2 添加环境变量，/PATH_TO_KBLAS_AND_KLAPACK为安装路径。

```
export LD_LIBRARY_PATH=/PATH_TO_KBLAS_AND_KLAPACK:$LD_LIBRARY_PATH
```

----结束

Fortran 接口迁移步骤

Fortran接口迁移与**C语言迁移步骤**一致。

3.3 FFTW 迁移到 KML_FFT

可替换性

所有接口均可以替换。

C 语言迁移步骤

用户可以通过以下两种方法将FFTW迁移到KML_FFT。

第一种：直接迁移（需要修改部分代码）。

 版权所有 © 华为技术有限公司

步骤1 使用全局替换，将带有fftw前缀的接口替换成kml_fft前缀，将带有fftwf前缀的接口替换成kml_fftf前缀，如果有使用到FFTW头文件定义的宏定义常量，也需要将FFTW前缀替换成KML_FFT前缀。将#include <fftw.h>替换成#include <kfft.h> 。

步骤2 在编译选项中将-lfftw/-lfftwf替换为-lkfft/-lkfftf。

步骤3 添加环境变量，/PATH_TO_KFFT表示KML_FFT的安装路径。

```
export LD_LIBRARY_PATH=/PATH_TO_KFFT:$LD_LIBRARY_PATH
```

----结束

第二种：通过适配层迁移（不需要修改源码，推荐使用该方法）

步骤1 添加环境变量，应用适配层so，替换原来软件中链接的fftw库。

```
export LD_LIBRARY_PATH=/PATH_TO_KML_ADAPTER/:$LD_LIBRARY_PATH
```

步骤2 添加环境变量，使用kfft链接到适配层。

```
export LD_LIBRARY_PATH=/PATH_TO_KML_FFT/:$LD_LIBRARY_PATH
```

----结束

Fortran 接口迁移步骤

FFTW包含f77，f90，f03接口，其中f77接口定义与KML_FFT提供的Fortran接口不同，但内部逻辑实现基本相同。需要在KML_FFT外层封装一层适配f77的头文件。

3.4 Netlib LAPACK 迁移到 KML_LAPACK

可替换性

LAPACK采用与Netlib LAPACK完全一致的Fortran接口。

表3-2为可替换的接口与Netlib LAPACK的Fortran接口的映射关系（仅给出已优化的接口）。

表 3-2 Netlib LAPACK 的替换映射关系

Netlib接口	KML对应接口
sgetrf	sgetrf
dgetrf	dgetrf
cgetrf	cgetrf
zgetrf	zgetrf
sgeqrf	sgeqrf
dgeqrf	dgeqrf
cgeqrf	cgeqrf
zgeqrf	zgeqrf
sgerqf	sgerqf

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
dgerqf	dgerqf
cgerqf	cgerqf
zgerqf	zgerqf
sgeqlf	sgeqlf
dgeqlf	dgeqlf
cgeqlf	cgeqlf
zgeqlf	zgeqlf
sgelqf	sgelqf
dgelqf	dgelqf
cgelqf	cgelqf
zgelqf	zgelqf
sppsv	sppsv
dppsv	dppsv
cppsv	cppsv
zppsv	zppsv
sptsv	sptsv
dptsv	dptsv
cptsv	cptsv
zptsv	zptsv
sgetri	sgetri
dgetri	dgetri
cgetri	cgetri
zgetri	zgetri
ssyevd	ssyevd
dsyevd	dsyevd
cheevd	cheevd
zheevd	zheevd
ssytrd	ssytrd
dsytrd	dsytrd
chetrd	chetrd

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
zhetrd	zhetrd
spotrf	spotrf
dpotrf	dpotrf
cpotrf	cpotrf
zpotrf	zpotrf
spttrf	spttrf
dpttrf	dpttrf
cpttrf	cpttrf
zpttrf	zpttrf
spotri	spotri
dpotri	dpotri
cpotri	cpotri
zpotri	zpotri
sgesv	sgesv
dgesv	dgesv
cgesv	cgesv
zgesv	zgesv
sorglq	sorglq
dorglq	dorglq
cunglq	cunglq
zunglq	zunglq
sormlq	sormlq
dormlq	dormlq
cunmlq	cunmlq
zunmlq	zunmlq
sorgql	sorgql
dorgql	dorgql
cungql	cungql
zungql	zungql
sormql	sormql

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
dormql	dormql
cunmql	cunmql
zunmql	zunmql
sorgqr	sorgqr
dorgqr	dorgqr
cungqr	cungqr
zungqr	zungqr
sorgrq	sorgrq
dorgrq	dorgrq
cungrq	cungrq
zungrq	zungrq
sormrq	sormrq
dormrq	dormrq
cunmrq	cunmrq
zunmrq	zunmrq
sormqr	sormqr
dormqr	dormqr
cunmqr	cunmqr
zunmqr	zunmqr
ssyev	ssyev
dsyev	dsyev
cheev	cheev
zheev	zheev
spttrs	spttrs
dpttrs	dpttrs
cpttrs	cpttrs
zpttrs	zpttrs
sptts2	sptts2
dptts2	dptts2
cptts2	cptts2

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
zptts2	zptts2
slasr	slasr
dlasr	dlasr
clasr	clasr
zlasr	zlasr
sgtsv	sgtsv
dgtsv	dgtsv
cgtsv	cgtsv
zgtsv	zgtsv
sgttrf	sgttrf
dgttrf	dgttrf
cgttrf	cgttrf
zgttrf	zgttrf
sgttrs	sgttrs
dgttrs	dgttrs
cgttrs	cgttrs
zgttrs	zgttrs
ssytrd_2stage	ssytrd_2stage
dsytrd_2stage	dsytrd_2stage
chetrd_2stage	chetrd_2stage
zhetrd_2stage	zhetrd_2stage
strtrs	strtrs
dtrtrs	dtrtrs
ctrtrs	ctrtrs
ztrtrs	ztrtrs
slaset	slaset
dlaset	dlaset
claset	claset
zlaset	zlaset
ssptrf	ssptrf

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
dsptrf	dsptrf
csptrf	csptrf
zsptrf	zhptrf
chptrf	chptrf
zhptrf	zsptrf
spptrf	spptrf
dpptrf	dpptrf
cpptrf	cpptrf
zpptrf	zpptrf
spptrs	spptrs
dpptrs	dpptrs
cpptrs	cpptrs
zpptrs	zpptrs
spptri	spptri
dpptri	dpptri
cpptri	cpptri
zpptri	zpptri
sgetrs	sgetrs
dgetrs	dgetrs
cgetrs	cgetrs
zgetrs	zgetrs
sposv	sposv
dposv	dposv
cposv	cposv
zposv	zposv
strtri	strtri
dtrtri	dtrtri
ctrtri	ctrtri
ztrtri	ztrtri
slaswp	slaswp

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
dlaswp	dlaswp
claswp	claswp
zlaswp	zlaswp
slascl	slascl
dlascl	dlascl
clascl	clascl
zlascl	zlascl
slange	slange
dlange	dlange
clange	clange
zlange	zlange
dsgesv	dsgesv
zcgesv	zcgesv
dgels	dgels
sgels	sgels
cgels	cgels
zgels	zgels
dgelsd	dgelsd
sgelsd	sgelsd
cgelsd	cgelsd
zgelsd	zgelsd
dgelss	dgelss
sgelss	sgelss
cgelss	cgelss
zgelss	zgelss
csteqr	csteqr
dsteqr	dsteqr
ssteqr	ssteqr
zsteqr	zsteqr
cgtts2	cgtts2

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
dgtts2	dgtts2
sgtts2	sgtts2
zgtts2	zgtts2
dormbr	dormbr
sormbr	sormbr
cunmbr	cunmbr
zunmbr	zunmbr
sorgtr	sorgtr
dorgtr	dorgtr
cungtr	cungtr
zungtr	zungtr
dlacpy	dlacpy
slacpy	slacpy
clacpy	clacpy
zlacpy	zlacpy
dstedc	dstedc
sstedc	sstedc
cstedc	cstedc
zstedc	zstedc
dgesvd	dgesvd
sgesvd	sgesvd
cgesvd	cgesvd
zgesvd	zgesvd
dgebrd	dgebrd
sgebrd	sgebrd
cgebrd	cgebrd
zgebrd	zgebrd
dbdsdc	dbdsdc
sbdsdc	sbdsdc
dgesdd	dgesdd

 版权所有 © 华为技术有限公司

Netlib接口	KML对应接口
sgesdd	sgesdd
cgesdd	cgesdd
zgesdd	zgesdd
dbdsqr	dbdsqr
sbdsqr	sbdsqr
cbdsqr	cbdsqr
zbdsqr	zbdsqr

以上接口均为Fortran接口，在C语言中使用时需在接口名称后增加下划线，且参数类型均为指针类型，请参见《Kupeng HPCKit 26.1.RC1开发指南》中“KML_LAPACK函数定义”章节。

C 语言迁移步骤

步骤1 KML LAPACK提供的Fortran接口与Netlib LAPACK完全一致，C语言调用时可通过#include "klapack.h" 获得接口声明。

----结束

其他代码无需修改。

Fortran 接口迁移步骤

Fortran接口与Netlib LAPACK一致，代码无需修改。

3.5 PETSc 迁移到 KML_SOLVER

3.5.1 单机版

可替换性

当前KML_SOLVER仅有单节点多线程，无法替代PETSc的多节点功能。

如表 PETSc的替换映射关系所示为可替换的接口与PETSc相应接口的映射关系（仅给出已优化的接口）。

表 3-3 PETSc 的替换映射关系

PETSc接口	KML_SOLVER接口
KSPCreate	KmlIssCgInitDI
KSPSolve	KmlIssCgSolveDI

 版权所有 © 华为技术有限公司

PETSc接口	KML_SOLVER接口
KSPDestroy	KmlIssCgCleanDI

以上接口均为C接口，请参见《Kunpeng HPCKit 26.1.RC1 安装指南》中“鲲鹏数学库开发指南 > 应用数学库 > KML_SOLVER库函数说明 > 函数定义（单机版）”章节。

C 语言迁移步骤

PETSc提供C接口，KML_SOLVER同样也提供了C接口，但接口名及接收参数不一致，需要进行对应功能的切换，举例如下：

1. 初始化。

   迁移前：

   ```
   KSP            ksp;
   PetscErrorCode ierr;
   ierr = KSPCreate(PETSC_COMM_WORLD,&ksp);
   ierr = KSPSetOperators(ksp,A,A);
   ```

   迁移后：

   ```
   int n = 8;
   int ia[9];
   int ja[17];
   double a[17];
   void* handle;
   int error = KmlIssCgInitDI(&handle, &n, a, ja, ia);
   ```

2. 求解。

   迁移前：

   ```
   Vec            u,x,b;
   ierr = VecCreate(PETSC_COMM_WORLD,&u);
   ierr = VecSetSizes(u,PETSC_DECIDE,rdim);
   ierr = VecSetFromOptions(u);CHKERRQ(ierr);
   ierr = VecDuplicate(u,&b);CHKERRQ(ierr);
   ierr = VecDuplicate(b,&x);CHKERRQ(ierr);

   ierr = KSPSolve(ksp,b,x);CHKERRQ(ierr);
   ```

   迁移后：

   ```
   double b[8];
   /* Solution vector */
   double x[8];
   int nrhs = 1; /* Number of right-hand sides */
   int ldx=n, ldb=n; /* Leading dimension of B and X */
   int error = KmlIssCgSolveDI(&handle, &nrhs, x, &ldx, b, &ldb);
   ```

3. 获取迭代参数。

   迁移前：

   ```
   PetscInt       its;
   ierr = KSPGetIterationNumber(ksp,&its);CHKERRQ(ierr);
   ```

   迁移后：

   ```
   int *data;  // user data.
   int nd = 1;
   int error = KmlIssCgGetDII(&handle, ITERATION_COUNT, data, nd);
   ```

4. 清除。

   迁移前：

   ```
   ierr = KSPDestroy(&ksp);
   ```

   迁移后：

   ```
   int error = KmlIssCgCleanDI(&handle);
   ```

 版权所有 © 华为技术有限公司

5. 头文件。

   迁移前：

   #include "petscksp.h"

   迁移后：

   #include "kml_iss.h"

6. 编译链接。

   替换PETSc相关的链接选项，具体选项请参见《Kunpeng HPCKit 26.1.RC1 安装指南》安装KML。

3.5.2 集群版

可替换性

当前KML_SCASOLVER支持多节点多进程多线程，可以替代PETSc的多节点多进程功能。

如表 PETSc的替换映射关系所示为可替换的接口与PETSc相应接口的映射关系（仅给出已优化的接口）。

表 3-4 PETSc 的替换映射关系

PETSc接口	KML_SCASOLVER接口
KSPCreate	KmlScaissCgInitStripesDI
KSPSetTolerances	KmlScaissCgSetDII
KSPSolve	KmlScaissCgSolveDI
KSPGetIterationNumber/ KSPGetResidualNorm	KmlScaissCgGetDII/ KmlScaissCgGetDID
KSPDestroy	KmlScaissCgCleanDI

以上接口均为C接口，请参见《Kunpeng HPCKit 26.1.RC1 开发指南》中“鲲鹏数学库开发指南 > 应用数学库 > KML_SOLVER库函数说明 > 函数定义（集群版）”章节。

C 语言迁移步骤

第一种：直接迁移

PETSc提供C接口，KML_SCASOLVER同样也提供了C接口，但接口名及接收参数不一致，需要进行对应功能的切换，举例如下：

1. 初始化。

   迁移前：

   ```
   KSP         ksp;
   PetscErrorCode ierr;
   ierr = KSPCreate(PETSC_COMM_WORLD,&ksp);
   ierr = KSPSetOperators(ksp,A,A);
   ```

   迁移后：

   ```
   int n = 8;
   int nstripes = 1;
   ```

 版权所有 © 华为技术有限公司

```
int stripeWidth[1];
int stripeRow[1];
int ia[9];
int ja[17];
double a[17];
KmlScasolverTask* handle;
int err;
err = KmlScaissCgInitStripesDI(&handle, n, nstripes , stripeWidth, stripeRow, a, ja, ia,
MPI_COMM_WORLD);
```

2. 设定求解参数

   迁移前：

   ```
   ierr = KSPSetTolerances(ksp,1e-4,PETSC_DEFAULT,PETSC_DEFAULT,2000);
   ```

   迁移后：

   ```
   double eps = 1e-4;
   err = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1); assert(err == 0);
   int max_iters = 2000;
   err = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
   ```

3. 求解。

   迁移前：

   ```
   Vec            u,x,b;
   ierr = VecCreate(PETSC_COMM_WORLD,&u);
   ierr = VecSetSizes(u,PETSC_DECIDE,rdim);
   ierr = VecSetFromOptions(u);CHKERRQ(ierr);
   ierr = VecDuplicate(u,&b);CHKERRQ(ierr);
   ierr = VecDuplicate(b,&x);CHKERRQ(ierr);

   ierr = KSPSolve(ksp,b,x);CHKERRQ(ierr);
   ```

   迁移后：

   ```
   int nb=1;
   double* x[8];
   double* b[8];
   int ldx=4;
   int ldb=4;
   err = KmlScaissCgSolveDI(&handle, nb, x, ldx, b, ldb);
   ```

4. 获取迭代参数。

   迁移前：

   ```
   PetscInt       its;
   PetscScalar      norm;
   ierr = KSPGetIterationNumber(ksp,&its);CHKERRQ(ierr);
   ierr = KSPGetResidualNorm(ksp,&norm);CHKERRQ(ierr);
   ```

   迁移后：

   ```
   int idata[] = {0};
   err = KmlScaissCgGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
   double ddata[] = {0};
   err = KmlScaissCgGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
   ```

5. 清除。

   迁移前：

   ```
   ierr = KSPDestroy(&ksp);
   ```

   迁移后：

   ```
   err = KmlScaissCgCleanDI(&handle);
   ```

6. 头文件。

   迁移前：

   #include "petscksp.h"

   迁移后：

   #include "kml_scaiss.h"

7. 编译链接。

 版权所有 © 华为技术有限公司

替换PETSc相关的链接选项，具体选项请参见《Kunpeng HPCKit 26.1.RC1 安装指南》安装KML。

第二种：通过适配层迁移

步骤1 将#include "petscksp.h"改为#include "petscksp_kp.h"，MatCreateMPIAIJWithArrays与VecCreateMPIWithArray接口需要在第二个参数位置新增参数ilower（当前进程上矩阵首行对应的全局下标）。

步骤2 通过加载HPCkit相关环境变量，设置PETSc的头文件路径和KML适配层动态库路径，环境变量的具体加载方式参见《Kunpeng HPCKit 26.1.RC1 安装指南》中“设置环境变量”章节，设置成功后重新编译应用。

```
mpicc program.c -lpetsc_kp -lkscasolver -lkservice -lklapack -lkblas -lkm -lm
```

----结束

*实际应用场景中往往需要更加灵活的调用，可参考适配层的内部实现：petsc_kunpeng.c

表 3-5 当前已适配接口

PETSc接口
PetscInitialize
PetscFinalize
MatCreateMPIAIJWithArrays
VecCreateMPIWithArray
KSPCreate
KSPSetOperators
KSPSetType
KSPSetTolerances
KSPSetUp
KSPSolve
KSPGetIterationNumber
KSPGetResidualNorm
KSPGetPC
PCSetType
KSPDestroy
VecDestroy
MatDestroy
VecGetArray

 版权所有 © 华为技术有限公司

3.6 HYPRE 迁移到 KML_SOLVER

3.6.1 集群版

迁移步骤

通过适配层迁移

步骤1 将#include "HYPRE.h"改为#include "HYPRE_kp.h"，去除其余HYPRE相关头文件，若涉及ILU作为预条件子使用，需要去除HYPRE_ILUCreate、HYPRE_ILUDestroy和HYPRE_ILUSetMaxIter等相关预条件子创建及设置接口，BoomerAMG作为预条件子同理。

步骤2 通过加载HPCkit相关环境变量，设置HYPRE的头文件路径和KML适配层动态库路径，环境变量的具体加载方式参见《Kunpeng HPCKit 26.1.RC1 安装指南》中“设置环境变量”章节，设置成功后重新编译应用

```
mpicc program.c  -lHYPRE_kp -lkscasolver -lkservice -lklapack -lkm -lm
```

----结束

*实际应用场景中往往需要更加灵活的调用，可参考适配层的内部实现：
hypre_kunpeng.c

表 3-6 当前已适配接口

HYPRE接口
HYPRE_Init
HYPRE_IJMatrixCreate
HYPRE_IJMatrixSetObjectType
HYPRE_IJMatrixInitialize
HYPRE_IJMatrixSetValues
HYPRE_IJMatrixAssemble
HYPRE_IJMatrixGetObject
HYPRE_IJVectorCreate
HYPRE_IJVectorSetObjectType
HYPRE_IJVectorInitialize
HYPRE_IJVectorSetValues
HYPRE_IJVectorAssemble
HYPRE_IJVectorGetObject
HYPRE_ParCSRBiCGSTABCreate

 版权所有 © 华为技术有限公司

HYPRE接口
HYPRE_ParCSRBiCGSTABSetTol
HYPRE_ParCSRBiCGSTABSetMaxIter
HYPRE_ParCSRBiCGSTABSetup
HYPRE_ParCSRBiCGSTABSolve
HYPRE_ParCSRBiCGSTABGetNumIterations
HYPRE_ParCSRBiCGSTABGetFinalRelativeResidualNorm
HYPRE_ParCSRBiCGSTABDestroy
HYPRE_ParCSRBiCGSTABSetPrecond
HYPRE_ParCSRGMRESCreate
HYPRE_ParCSRGMRESSetTol
HYPRE_ParCSRGMRESSetMaxIter
HYPRE_ParCSRGMRESSetup
HYPRE_ParCSRGMRESSolve
HYPRE_ParCSRGMRESGetNumIterations
HYPRE_ParCSRGMRESGetFinalRelativeResidualNorm
HYPRE_ParCSRGMRESDestroy
HYPRE_ParCSRGMRESSetPrecond
HYPRE_IJMatrixDestroy
HYPRE_IJVectorDestroy
HYPRE_Finalize

3.7 MUMPS 迁移到 KML_SOLVER

3.7.1 集群版

可替换性

用户通过给结构体中的变量赋值控制求解过程，可以通过给JOB参数赋不同的值控制MUMPS执行的具体操作，控制MUMPS求解功能的主要参数还包括数组ICNTL和CNTL。另一方面，用户可以通过结构体获取最中间或者最终的求解结果。当前KML_SCASOLVER支持多节点多进程多线程，可以替代MUMPS的多节点多进程功能。当前已实现double类型的MUMPS接口，JOB参数为-1、-2、1、2、3、6，支持一般矩阵、对称正定矩阵和一般对称矩阵。

 版权所有 © 华为技术有限公司

表 3-7 MUMPS 的替换映射关系

MUMPS接口	KML_SOLVER接口
id.job=JOB_INIT; dmumps_c(&id);	KmlScadssInit
id.job=1; dmumps_c(&id);	KmlScadssAnalyze
id.job=2; dmumps_c(&id);	KmlScadssFactorize
id.job=3; dmumps_c(&id);	KmlScadssSolve
id.job=JOB_END; dmumps_c(&id);	KmlScadssClean

迁移步骤

通过适配层迁移

MUMPS提供C接口，KML_SCASOLVER同样也提供了C接口，直接进行适配层的迁移。

步骤1 将#include "dmumps_c.h"改为#include "MUMPS_kp.h"。

步骤2 通过加载HPCkit相关环境变量，设置KML求解器适配器的头文件MUMPS_kp.h所在路径和kscasolver动态库路径，具体加载方式可参见《Kunpeng HPCKit 26.1.RC1 安装指南》中“设置环境变量”章节，然后在重新编译应用。

```
mpicc program.c  -ldmumps_kp  -lkscasolver -lkservice -lklapack  -lkblas  -lkm -lm
```

----结束

迁移前后，mumps的初始化接口与使用KML适配层的初始化接口不一样，mumps的初始化在传入A矩阵之前，而KML适配层需要在传入A矩阵之后。

*实际应用场景中往往需要更加灵活的调用，可参考适配层的内部实现：**dmumps_kunpeng.c**

迁移前：

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#include "dmumps_c.h"
int main ( int argc , char**argv ) {
   DMUMPS_STRUC_C id ;
   int n = 2;
   int64_t nnz = 2;
   int irn[] = { 1, 2 }; // MUMPS只支持COO存储格式
   int jcn[] = { 1, 2 };
   double *a = (double *)malloc(2*sizeof(double));
   double *rhs = (double *)malloc(2*sizeof(double));
   //double a[2];
   //double rhs[2];
   double sol[] = {0, 0};
   int myid, ierr;
   ierr = MPI_Init(&argc, &argv);
   ierr = MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   /*Define A and rhs */
   rhs[0] = 1.0;
   rhs[1] = 4.0;
   a[0] = 1.0;
   a[1] = 2.0;
   /*Initialize MUMPS instance.Use MPI COMM WORLD.*/
   id.job = JOB_INIT;
   id.par = 1;
```

 版权所有 © 华为技术有限公司

```
    id.sym = 0;
    id.comm_fortran = USE_COMM_WORLD;
    dmumps_c(&id);
    /*Define the problem on the host */
    if (myid == 0)
    {
        id.n = n;
        id.nnz = nnz;
        id.irn = irn;
        id.jcn = jcn;
        id.a = a;
        id.rhs = rhs;
    }

#define ICNTL(I) icntl[(I)-1] /*macro s.t.indices match documentation*/ /*No outputs */
    id.ICNTL(1) = -1;
    id.ICNTL(2) = -1;
    id.ICNTL(3) = -1;
    id.ICNTL(4) = 0;
    /*Call the MUMPS package.*/
    id.job = 6; //Analyze + Factorize + Solve
    dmumps_c(&id);
    id.job = JOB_END;
    dmumps_c(&id);
    /*Terminate instance */ if (myid == 0)
    {
        printf( " Solution is : (%8.2f %8.2f)\n", rhs[0], rhs[1]);
    }
    ierr = MPI_Finalize();
    return 0;
}
```

迁移后：

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#include "MUMPS_kp.h"
int main ( int argc , char**argv ) {
    DMUMPS_STRUC_C id ;
    int n = 2;
    int64_t nnz = 2;
    int irn[] = { 1, 2 }; // MUMPS只支持COO存储格式
    int jcn[] = { 1, 2 };
    double *a = (double *)malloc(2*sizeof(double));
    double *rhs = (double *)malloc(2*sizeof(double));
    //double a[2];
    //double rhs[2];
    double sol[] = {0, 0};
    int myid, ierr;
    ierr = MPI_Init(&argc, &argv);
    ierr = MPI_Comm_rank(MPI_COMM_WORLD, &myid);
    /*Define A and rhs */
    rhs[0] = 1.0;
    rhs[1] = 4.0;
    a[0] = 1.0;
    a[1] = 2.0;
    /*Initialize MUMPS instance.Use MPI COMM WORLD.*/
    id.job = JOB_INIT;
    id.par = 1;
    id.sym = 0;
    id.comm_fortran = USE_COMM_WORLD;
    if (myid == 0)
    {
        id.n = n;
        id.nnz = nnz;
        id.irn = irn;
        id.jcn = jcn;
        id.a = a;
        id.rhs = rhs;
    }
```

 版权所有 © 华为技术有限公司

```
    dmumps_c(&id); /*Define the problem on the host */

#define ICNTL(I) icntl[(I)-1] /*macro s.t.indices match documentation*/ /*No outputs */
    id.ICNTL(1) = -1;
    id.ICNTL(2) = -1;
    id.ICNTL(3) = -1;
    id.ICNTL(4) = 0;
    /*Call the MUMPS package.*/
    id.job = 6; //Analyze + Factorize + Solve
    dmumps_c(&id);
    id.job = JOB_END;
    dmumps_c(&id);
    /*Terminate instance */ if (myid == 0)
    {
        printf( " Solution is : (%8.2f %8.2f)\n", rhs[0], rhs[1]);
    }
    ierr = MPI_Finalize();
    return 0;
}
```

3.8 Netlib SCALAPACK 迁移到 KML_SCALAPACK

可替换性

SCALAPACK的Fortran接口与Netlib SCALAPACK完全一致。

如**表KML_SCALAPACK与Netlib SCALAPACK的替换映射关系**所示，该表给出了SCALAPACK中可替换的接口与Netlib SCALAPACK的Fortran接口的映射关系（仅给出已优化的接口）。

表 3-8 KML_SCALAPACK 与 Netlib SCALAPACK 的替换映射关系

Netlib接口	KML接口
psgetrf	psgetrf
pdgetrf	pdgetrf
pcgetrf	pcgetrf
pzgetrf	pzgetrf
pspotrf	pspotrf
pdpotrf	pdpotrf
pcpotrf	pcpotrf
pzpotrf	pzpotrf
psgeqrf	psgeqrf
pdgeqrf	pdgeqrf
pcgeqrf	pcgeqrf
pzgeqrf	pzgeqrf
psgels	psgels

版权所有 © 华为技术有限公司

Netlib接口	KML接口
pdgels	pdgels
pcgels	pcgels
pzgels	pzgels

以上接口均为Fortran接口，在C语言中使用时需在接口名称后增加下划线，且参数类型均为指针类型。请参见《Kunpeng HPCKit 26.1.RC1 开发指南》中“鲲鹏数学库开发指南 > 应用数学库 > KML_SCALAPACK库函数说明 > 函数定义”章节。

C 语言迁移步骤

步骤1 KML SCALAPACK提供的Fortran接口与Netlib SCALAPACK完全一致，C语言调用时可通过#include "kscalapack.h" 获得接口声明。

----结束

其他代码无需修改。

Fortran 接口迁移步骤

Fortran接口与Netlib SCALAPACK一致，代码无需修改。

3.9 Eigen 库使用 KML

Eigen 库介绍

Eigen库是一个用于线性代数运算的C++模板库。它提供了高性能的矩阵运算和线性代数操作，被广泛应用于科学计算、机器学习和计算机图形学等领域。详细介绍可参考**Eigen官网**。Eigen库官方提供了使能开源BLAS以及LAPACK的方式，同样可以使用到KML中的BLAS与LAPACK库，使用说明详见 **Using BLAS/LAPACK from Eigen** ，使用外部BLAS为Eigen提供了更高性能的矩阵运算接口。除了标准的BLAS、LAPACK接口，鲲鹏数学库提供了额外的BLAS接口供Eigen使用，详细接口信息请见**表1**。

Eigen 库 KML 优化适配

KML库在 **EIGEN_USE_BLAS** 机制基础上，进一步扩展并优化了多类Eigen接口，以提升应用场景下的整体性能。优化后的Eigen版本获取地址：**kunpeng-eigen**。

对于矩阵加法等基础算子，其使用方式与原生 Eigen 接口保持一致，仅需在编译阶段将宏定义 **-DEIGEN_USE_BLAS** 替换为 **-DKUNPENG_USE** 即可启用相应优化路径。针对其他已优化接口的具体使用方法以及验证流程，将在后续章节中进行详细说明。

下表给出了当前支持优化的 Eigen 功能范围。

 版权所有 © 华为技术有限公司

表 3-9 Eigen 库 KML 优化接口说明

接口名称	示例用法	适用数据类型	KML优化接口或方式
矩阵加法	1. C = A + B 2. C += A	float double std::complex<float> std::complex<double>	?omatadd ?imatadd
矩阵减法	1. C = A - B 2. C -= A	float double std::complex<float> std::complex<double>	?omatsub ?imatsub
逐元素乘	C = A.cwiseProduct(B)	float double std::complex<float> std::complex<double>	?omatmul
按行/列求和	v = A.rowwise().sum() v = A.colwise().sum()	float double std::complex<float> std::complex<double>	?gemv
连续子块拷贝	B = A.block(100, 200, 200, 300);	float double std::complex<float> std::complex<double>	?copy
逐元素拷贝	B = A.array()	float double std::complex<float> std::complex<double>	?copy
激活函数	B = A.tanh()	float	SVE intrinsic kernel

 版权所有 © 华为技术有限公司

接口名称	示例用法	适用数据类型	KML优化接口或方式
张量收缩	C=A.contract(B,dims)	2d float	sgemm

Eigen 库 TensorContraction 使能 KML

针对Eigen中的TensorContraction模块，在数据类型为float的二维张量收缩计算时，保持Eigen的多线程调度框架(ThreadPool)，替换底层算子，使能KML加速。

步骤1 获取优化后Eigen库。下载地址：**kunpeng-eigen**。

步骤2 进入eigen/eigen_blas目录下，此处为Eigen对接KML适配层，独立编译，以二进制方式连接到Eigen，编译命令：

```
clang -shared -fPIC -o libeigen_blas.so eigen_blas.c -L${PATH_TO_KBLAS} -lkblas
```

其中，PATH_TO_KBLAS需替换成对应的KML动态库路径，编译完成后生成libeigen_blas.so动态库。

步骤3 Eigen根目录下，设置环境变量OMP_NUM_THREADS=1，KUNPENG_OPT_CONTRACT=1，KUNPENG_CONTRACT_NUM_THREADS=目标线程数，再执行编译命令。

示例代码matmul_test.cpp：

```
#include <iostream>
#include <chrono>
#include <Eigen/Core>
#include <unsupported/Eigen/CXX11/Tensor>
#include <unsupported/Eigen/CXX11/ThreadPool>
using namespace Eigen;
int main() {
    int M = 5000, N = 1024, K = 1024;
    Tensor<float, 2, RowMajor> t_a(M, K);
    Tensor<float, 2, RowMajor> t_b(K, N);
    Tensor<float, 2, RowMajor> t_c(M, N);
    t_a.setRandom();
    t_b.setRandom();
    Eigen::array<Eigen::IndexPair<int>, 1> product_dims = { Eigen::IndexPair<int>(1, 0) };
    int num_threads = 4;
    Eigen::ThreadPool pool(num_threads);
    Eigen::ThreadPoolDevice device(&pool, num_threads);
    unsetenv("KUNPENG_OPT_CONTRACT");

    t_c.device(device) = t_a.contract(t_b, product_dims);
    auto start_ori = std::chrono::high_resolution_clock::now();
    t_c.device(device) = t_a.contract(t_b, product_dims);
    auto end_ori = std::chrono::high_resolution_clock::now();
    auto dur_ori = std::chrono::duration_cast<std::chrono::microseconds>(end_ori - start_ori).count();
    setenv("KUNPENG_OPT_CONTRACT", "1", 1);
    setenv("KUNPENG_CONTRACT_NUM_THREADS", "4", 1);
    t_c.device(device) = t_a.contract(t_b, product_dims);
    auto start_opt = std::chrono::high_resolution_clock::now();
    t_c.device(device) = t_a.contract(t_b, product_dims);
    auto end_opt = std::chrono::high_resolution_clock::now();

    auto dur_opt = std::chrono::duration_cast<std::chrono::microseconds>(end_opt - start_opt).count();
    std::cout << "--- MatMul (Contraction) KML Optimization Test ---" << std::endl;
    std::cout << "Scale          : " << M << "x" << N << "x" << K << std::endl;
    std::cout << "Original Time  : " << dur_ori << " us" << std::endl;
    std::cout << "Optimized Time : " << dur_opt << " us" << std::endl;
```

 版权所有 © 华为技术有限公司

```
    std::cout << "Speedup        : " << (float)dur_ori / dur_opt << "x" << std::endl;
    return 0;
}
```

编译命令：

```
clang++ matmul_test.cpp -I ./eigen -I ./eigen/eigen_blas -o matmul_test -march=armv9-a+sve -
DEIGEN_USE_THREADS -L ./eigen/eigen_blas -leigen_blas
```

成功输出如下所示：

```
--- MatMul (Contraction) KML Optimization Test ---
Scale          : 1024x1024x1024
Original Time  : 2131122 us
Optimized Time : 5400 us
Speedup        : 394.652x
```

----结束

Eigen 库 tanh 优化

在ARM架构上，Eigen tanh函数默认使用NEON指令集进行向量化处理。相比之下，SVE（可伸缩矢量扩展）支持128位到2048位的可变长度向量，可以一次性处理更多的tanh计算来提升性能。因此，在tanh的计算过程中，针对float数据类型，增加SVE intrinsic优化实现，以提升计算效率。

步骤1 获取优化后Eigen库。下载地址：**kunpeng-eigen**。

步骤2 进入eigen根目录下，编译以下示例测试代码：

tanh_test.cpp

```
#include <iostream>
#include <chrono>
#include <vector>
#include <Eigen/Core>
#include <unsupported/Eigen/CXX11/Tensor>
#include <unsupported/Eigen/CXX11/ThreadPool>
using namespace Eigen;
int main() {
    int M = 500, N = 500;
    int size = M * N;
    std::vector<float> input_raw(size);
    std::vector<float> output_raw(size);
    for(int i = 0; i < size; ++i) input_raw[i] = static_cast<float>(rand());
    TensorMap<Tensor<float, 2, RowMajor>> input_map(input_raw.data(), M, N);
    TensorMap<Tensor<float, 2, RowMajor>> output_map(output_raw.data(), M, N);
    Eigen::ThreadPool pool(1);
    Eigen::ThreadPoolDevice device(&pool, 1);
    auto start_ori = std::chrono::high_resolution_clock::now();
    output_map.device(device) = (1.0f * input_map).tanh();
    auto end_ori = std::chrono::high_resolution_clock::now();
    auto dur_ori = std::chrono::duration_cast<std::chrono::microseconds>(end_ori - start_ori).count();
    auto start_opt = std::chrono::high_resolution_clock::now();
    output_map.device(device) = input_map.tanh();
    auto end_opt = std::chrono::high_resolution_clock::now();
    auto dur_opt = std::chrono::duration_cast<std::chrono::microseconds>(end_opt - start_opt).count();
    std::cout << "--- Tanh SVE Optimization ---" << std::endl;
    std::cout << "Original Time  : " << dur_ori << " us" << std::endl;
    std::cout << "Optimized Time : " << dur_opt << " us" << std::endl;
    std::cout << "Speedup        : " << (float)dur_ori / dur_opt << "x" << std::endl;
    return 0;
}
```

编译命令：

```
clang++ test_tanh.cpp -I ./eigen -o test_tanh -march=armv9-a+sve -I ./eigen/eigen_blas -
DEIGEN_USE_THREADS
```

 版权所有 © 华为技术有限公司

执行后输出如下所示：

```
--- Tanh SVE Optimization ---
Original Time  : 43847 us
Optimized Time : 3987 us
Speedup        : 10.9975x
```

----结束

3.10 Numpy 使用 KML

Numpy 库介绍

Numpy（**Numerical Python**），是一个高性能Python库，支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库。KML_Numpy库是将KML数学库适配到Numpy的C源码中，使其相应函数的底层调用到KML接口，以此进一步提高性能。当前KML_Numpy适配的接口类型主要有：基础数学运算、FFT、线性代数。

环境要求

目前KML_Numpy适配了两个版本，分别为1.22.4和2.0.2，对构建环境有各自的要求：

- Python版本号为3.8~3.10，且含有开发模块。

  若Python为自行安装构建，一般在Python的安装目录下都会存在include目录包含了Python.h等头文件。若Python为系统自带，一般可以使用**yum**或**apt**命令安装Python开发包。安装命令如下：

  ```
  yum install python3-devel.aarch64    # 或 yum install python3-devel
  ```

  之后在“/usr/local/include”目录下出现“python3.x”或“python3”目录，内含开发Python相关功能会使用到的头文件。（示例路径为“/usr/local/include/python3”。）
- GCC版本号为8及以上。
- 其中**1.22.4**版本要求Cython版本号为0.29.xx，**2.0.2**版本要求Cython>=3.0.6，meson-python>=0.15.0。
- 已安装KML。

构建 KML_Numpy 库

以下为**1.22.4**版本Numpy构建方式：

步骤1 安装1.22.4版本开源Numpy库。

```
pip3 install numpy==1.22.4
```

步骤2 安装0.29.xx版本cython库。

```
pip3 install cython==0.29.34
```

步骤3 下载kml_adapter，并上传到构建环境（假设上传至“/home”目录下）

下载地址：**https://atomgit.com/openeuler/kml_adapter**

步骤4 配置相关环境变量（假设HPCKit安装路径为/opt/HPCKit）。

```
export KML_PATH=/opt/HPCKit/latest/kml/gcc
export PY3_HEADER_PATH=/usr/local/include/python3
export OPT=" -O3 -fopenmp -fopenmp-simd -I numpy/linalg -DHAVE_NEON_NPY -DHAVE_HUAWEI_KML -DNDEBUG -mtune=tsv110 -fno-math-errno -I ${PY3_HEADER_PATH} -I /usr/local/include/ -I ${KML_PATH}/
```

 版权所有 © 华为技术有限公司

```
include -L ${KML_PATH}/lib/neon -L ${KML_PATH}/lib/noarch -L ${KML_PATH}/lib/neon/kblas/multi -L ${KML_PATH}/lib/noarch/kvml/single -L ${KML_PATH}/lib/neon/kspblas/single -Wl,-rpath=${KML_PATH}/lib/neon -Wl,-rpath=${KML_PATH}/lib/noarch -Wl,-rpath=${KML_PATH}/lib/neon/kblas/multi -Wl,-rpath=${KML_PATH}/lib/noarch/kvml/single -Wl,-rpath=${KML_PATH}/lib/neon/kspblas/single -lkblas -lkvml -lkm -lkspblas -lklapack -lknpy -lkfft -lkfftf -lkm_l9 -lm -lksvml -lgfortran"
```

步骤5 编译。

```
cd /home/kml_adapter/numpy_adapter
NPY_BLAS_ORDER= NPY_LAPACK_ORDER= python3 setup.py build -j16
```

步骤6 确认编译是否成功。

如果编译成功，将在“numpy_adapter/build/”下根据你的环境生成如下几个目录：

```
lib.linux-aarch64-3.x
src.linux-aarch64-3.x
temp.linux-aarch64-3.x
```

----结束

以下为**2.0.2**版本Numpy构建方式：

步骤1 下载Numpy-2.0.2源码。

下载地址：**numpy_kml**

步骤2 下载子程序。

```
git submodule update --init --recursive
```

步骤3 进入根目录执行以下命令。

```
sed -i "s|\$PREFIX|$(pwd)|g" pyproject.toml
```

步骤4 添加环境变量为pcfiles所在路径。

```
export PKG_CONFIG_PATH=$PWD/pcfiles
```

步骤5 执行编译命令。

```
python -m build
```

----结束

Numpy 库使能 KML

1.22.4版本Numpy使能：

配置环境变量，确保优先调用KML_Numpy。（假设构建生成的是“kml_adapter/numpy_adapter/build/lib.linux-aarch64-3.10”。）

```
export PYTHONPATH=/home/kml_adapter/numpy_adapter/build/lib.linux-aarch64-3.10:$PYTHONPATH
```

2.0.2版本Numpy使能：

上述步骤构建成功会在dist目录下生成一个.whl文件。（假设构建生成的是“numpy-2.0.2-cp39-cp39-linux_aarch64.whl”）

```
pip install numpy-2.0.2-cp39-cp39-linux_aarch64.whl
```

 版权所有 © 华为技术有限公司

4 修订记录

表 4-1

发布日期	修订记录
2026-06-30	第一次正式发布。

 版权所有 © 华为技术有限公司